Clarity Vision

This directory captures the goals, architecture, and rationale for the next Clarity: a redesigned, independently specified language that compiles to Wasm. It is the first source language for a new chain that executes Wasm.

These documents are working drafts intended to seed discussion. They are expected to eventually move to a dedicated language repository.

Documents

Document	Purpose
01-why-clarity.md	Why a decidable language on a Wasm chain: what the VM provides, what only the language can provide, the honest gaps, and why not just compile a general-purpose language
02-goals.md	Goals, non-goals, and success criteria
03-architecture.md	The Clarity toolchain, the module format and chain ABI, the guarantees the chain enforces for every module, and the compiler trust path
04-spec-plan.md	Plan for the language specification, the chain ABI it depends on, and the conformance suite
05-roadmap.md	Phased execution plan with milestones, dependencies, and risks
06-issue-tracker.md	Disposition of every open `clarity` issue against the spec draft, and candidate additions
spec/	The specification working draft: a minimal core language plus a standard library written in the language itself

One-paragraph summary

The new chain executes Wasm modules, meters them in fuel, and enforces a set of runtime guarantees for every module regardless of source language (03-architecture.md section 5). Wasm cannot give the guarantees that made Clarity worth having, because Wasm is Turing-complete: fuel makes execution *stop*, but it does not make the worst case *known in advance*. Clarity adds that. Every Clarity program terminates by construction, and every loop and value is bounded by its type, so a contract's worst-case cost is computable before it runs, whole-program analysis is complete, and safety proofs are tractable. This effort (1) specifies Clarity spec-first as a small core language plus a standard library written in it, (2) builds a deterministic compiler to Wasm held to the spec by a conformance suite, (3) defines, with the chain, an ABI that every source language shares, and (4) builds the toolchain that turns decidability into products: source verification, a static cost analyzer, and a property checker.

Clarity 1–6 contracts on the existing chain keep running on the interpreter, and the interpreter and the Wasm engine can call each other. That arrangement is outside the scope of these documents.

Why Clarity on a Wasm Chain

The new chain executes Wasm, and any language that compiles to Wasm can target it. So the question is no longer "why not switch VMs". It is "why a dedicated contract language, instead of Rust or another general-purpose language compiled to the same Wasm". This document makes the case, and states honestly what the Wasm-first design costs.

1. What the VM gives and what only the language can give

Wasm gives the chain a mature, sandboxed, portable execution engine, fuel metering, good performance, and a path to more than one source language. The chain adds runtime guarantees on top that hold for every module (section 1.4).

Wasm cannot give **decidability**. A Wasm module can loop without bound; fuel guarantees that execution stops, not that anyone can know beforehand how much it will cost or what it can do. Those properties have to come from the source language.

1.1 Decidability

Clarity is deliberately **not Turing-complete**:

- There is no recursion (direct or mutual) and no unbounded loop construct. Iteration exists only as `map`, `filter`, `fold-left`, and `fold-right` over sequences whose maximum length is fixed by their type.
- Every type has a statically known maximum size. Lists, buffers, and strings all carry length bounds; there are no unbounded data structures.
- Call depth is bounded and contracts cannot be re-entered.

Consequence: **every Clarity program provably terminates**, and every execution path is statically enumerable in principle. The halting problem, which makes complete static analysis impossible for general Wasm, does not apply to a module compiled from Clarity.

1.2 What decidability enables

These capabilities justify the language, and this effort commits to building them:

- **Sound static cost bounds.** Because all loops and data are bounded by types, the worst-case fuel of a function's own code is computable before deployment without simulation, as a function of its arguments' sizes (a list's actual length, not only its type bound, so the bound for a one-element list is not the bound for a thousand). A wallet can display a true maximum fee; a block producer can pack against guaranteed bounds; there is no "estimate by simulating and hope state doesn't change" pattern. The bound composes across `contract-call?` only as far as the callees are themselves Clarity with verified source. A call through a trait, or into a module written in another

language, can consume the whole remaining budget, and the analyzer reports it as such (03-architecture.md section 5).
- **Complete static analysis.** Whole-program analyses (taint tracking, asset flow, authorization checks) terminate and can be precise, not heuristic. An analyzer can enumerate every path by which a contract's assets can move.
- **Tractable safety proofs.** All functions are total, state access is explicit, and there is no reentrancy, so verification conditions stay inside decidable or well-behaved SMT fragments. Proving invariants such as supply conservation, access control, and "no path locks funds" is an engineering task, not a research program.

1.3 The deployed artifact is Wasm

The chain stores and executes Wasm. It never sees Clarity source. This is the price of a chain that more than one language can target, and it changes one of legacy Clarity's claims: legacy Clarity had no compiler between the author and the chain, and new Clarity does.

A compiler in the trust path means a compiler bug can make the deployed code differ from the source everyone audited. The plan pays that down directly:

- **A specification and conformance suite** define what a correct compiler produces, independent of any one implementation.
- **Deterministic, reproducible builds.** The same source and compiler version produce a byte-identical module, and each module records which compiler built it and the hash of its source. Anyone can rebuild and compare.
- **Source verification as a product.** Analyses, cost bounds, and proofs are computed on source and attach to a verified (source, compiler, module) triple. They apply to the deployed code exactly when that triple checks.
- **A compiler that is small by design.** The core language is about a dozen forms and eighty primitives with no bespoke evaluation rules, and the standard library is compiled like user code. Validating the translation of a first-order, loop-bounded language is far more tractable than for a general-purpose one, which keeps verified compilation a realistic long-term goal.

1.4 Safety by construction

Two layers enforce safety, and it matters which one enforces what.

The **chain** enforces these for every module, whatever its source language, so they hold even when a Clarity contract calls one written in something else (the normative table is 03-architecture.md section 5):

- **No reentrancy**, the largest exploit class in EVM history, and bounded call depth.
- **Read-only frames**: a write attempted inside a read-only call is rejected at runtime.
- **Rollback on error**: a public function that returns `err` has its state changes discarded.
- **Well-typed boundaries**: every value crossing a contract call is validated against the callee's declared interface.

- **Post-conditions and asset allowances**: the sender bounds what a transaction may do to their assets, and `as-contract?` bounds what a contract may do with its own, regardless of the called code.

The **language** adds, for Clarity contracts:

- **No undefined behavior**: arithmetic overflow aborts deterministically; there are no null references.
- **Errors must be handled**: ignoring a `response` is a type error.
- **Decidability**, with everything in 1.2.

1.5 Bitcoin-native

Clarity contracts read Bitcoin chain state directly (burn block info, Bitcoin transaction outputs, Merkle inclusion proofs). The chain ABI exposes that state to every language; Clarity makes it first-class in the type system and standard library.

2. Honest ledger

Gap	Reality	Plan item
No specification yet	The design exists as a working draft (spec/)	Spec-first design (04-spec-plan.md)
Compiler in the trust path	Deployed code is only as trustworthy as the compiler that produced it	Conformance suite, reproducible builds, source verification (03-architecture.md)
Guarantees are per-language	A module from another language has the chain's runtime guarantees but not Clarity's static ones	Verified-source labeling in wallets and explorers, so users know which they are looking at
Properties unexploited	No static cost analyzer or prover exists yet	Flagship toolchain deliverables (05-roadmap.md)
Not source-compatible with Clarity 1–6	Builders must port contracts	Disposition table for every legacy construct (spec/04-standard-library.md)
Mixed ecosystem for years	Tokens, PoX, and sBTC are legacy contracts; a new contract that calls them crosses a type mapping between legacy analysis types and the chain ABI	Named as a chain-owned deliverable with the mismatches listed (03-architecture.md section 7)
Cost bounds stop at foreign code	A call through a trait or into a non-Clarity module has no static bound	Bounds are stated per verified-Clarity call graph, and the analyzer marks the boundary (05-roadmap.md risk 7)
Developer experience	Tooling, docs, and error messages behind mainstream ecosystems	A stable spec and toolchain crates as the foundation for tools

Each gap is an engineering problem. None is a fundamental limitation. The reverse does not hold for general-purpose languages: their deficits relative to Clarity are fundamental.

3. Why not a general-purpose language on the same Wasm

Option	What you gain	What you permanently lose
Rust (or C, Go, AssemblyScript) to Wasm	Mature toolchains, large developer pool, raw expressiveness	Decidability; static cost bounds; complete analysis; tractable proofs. Contract safety reduces to library discipline, and the language's compiler and runtime become a large trust surface
Solidity or an EVM-style language to Wasm	Familiarity for EVM developers	All of the above, plus a design built around a reentrancy-permitting call model
Move-style resource language	Linear resource types (genuinely good)	Still Turing-complete: termination, exact costs, and complete analysis remain impossible

These are not either/or. The chain is multi-language by design, and nothing stops other languages from targeting it. Clarity's role is to be the language in which the properties above are guaranteed, and the one that consensus-critical and high-value contracts are written in.

The pitch, in one sentence:

> **The contract language where the worst case is a theorem, not a hope.** Every contract's maximum cost is known before it runs, its safety properties are machine-checkable, and anyone can verify that the code on chain was built from the code they read.

4. The commitment this implies

A decidable language only earns its place if we ship what decidability makes possible. Concretely (see 05-roadmap.md):

1. A **specification**, designed spec-first.
2. A **deterministic compiler** to Wasm, pinned to the spec by a **conformance suite**.
3. **Reproducible builds and source verification**, so analyses of source apply to deployed code.
4. A **static cost analyzer** producing sound worst-case fuel bounds per function.
5. **Proof-friendly semantics** and a verification toolchain path.

Goals and Non-Goals

Goals

G1 — Clarity is a specified language compiled to Wasm

- Clarity is designed **spec-first**: grammar, type system, static and dynamic semantics, compilation to Wasm, and the standard library, each normative and each backed by conformance tests.
- The reference compiler implements the spec and is validated against it, never the reverse. Any other Clarity compiler is held to the same suite.
- The toolchain lives in its own repository, published as crates with their own CI, release cadence, and versioning policy.

G2 — One chain ABI, shared by every source language

- The chain defines a single ABI: the module format, interface metadata, value encoding, host functions, call semantics, and fuel schedule. Clarity is its first consumer, not its owner.
- Nothing on chain is Clarity-specific. The chain never parses Clarity, and a Clarity contract has no powers on chain that a module from another language lacks.
- The ABI is validated by Clarity and by at least one other source-language prototype before it is frozen, so it is not accidentally Clarity-shaped.

G3 — Runtime guarantees for every module

The chain enforces a fixed set of guarantees for every module, whatever language produced it: fuel limits, no reentrancy, read-only frames, rollback on `err`, boundary validation, and asset allowances. The normative list, with which layer enforces each, is 03-architecture.md section 5 and is not repeated here.

G4 — Cash in decidability

The properties unique to Clarity become shipped products:

- **Static cost analyzer**: sound worst-case fuel bounds (plus host charges) per public function, computed before deployment. Bounds are functions of argument sizes, not only of argument types, so they are tight enough to display as a fee. They cover the contract's own code and, transitively, callees that are verified Clarity; at a trait call or a call into another language the analyzer reports the remaining budget as the bound. The foundation for true max-fee display, simulation-free fee estimation, and block packing.
- **Proof-friendly semantics**: written so that verification conditions land in decidable or well-supported SMT fragments; a contract-property checker (supply conservation, authorization invariants, no fund lockup) is a first-class roadmap item.

G5 — A trustworthy compiler path

- **Deterministic builds**: the same build inputs produce a byte-identical module on every platform. The inputs are the source, the compiler version, the interface of every contract the source calls by literal, and the deploying principal when deployer-relative names are resolved at compile time (03-architecture.md section 2).
- **Provenance**: each module records its compiler version, its source hash, and a hash of every callee interface it was compiled against.
- **Source verification**: anyone can rebuild a deployed module from its claimed source and the recorded inputs and compare. Analyses and proofs attach to a verified (source, build inputs, module) triple.

Non-Goals

- **N1 — Source compatibility with Clarity 1–6.** Rejected. The new language has no compatibility obligation in syntax or semantics. The disposition table in spec/04-standard-library.md is the porting guide. Legacy contracts keep running on the interpreter, with interop handled by the chain outside these documents.
- **N2 — Turing completeness.** Decidability is the product. Any proposal that introduces unbounded loops, recursion, or unbounded data is out of scope by definition.
- **N3 — On-chain compilation.** The chain stores and executes Wasm. It does not parse, type-check, or compile Clarity.
- **N4 — Being the only language.** The chain is multi-language. Clarity competes on guarantees, not on privileges.
- **N5 — A general-purpose language.** Clarity stays a smart contract language: no file I/O, FFI, or off-chain execution.

Success criteria

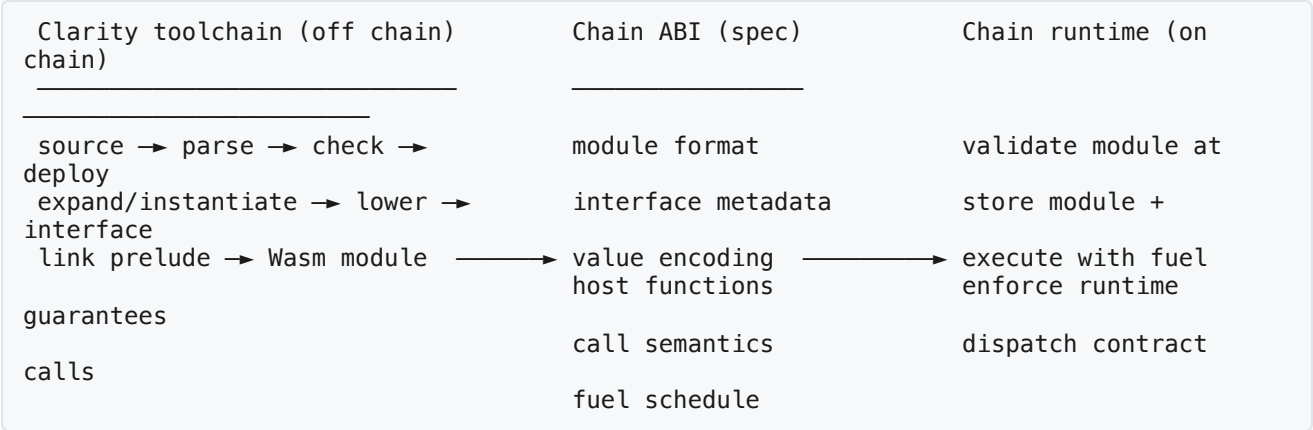
1. A complete, versioned spec for the language, with a conformance suite.
2. A reference compiler that passes the suite and produces byte-identical modules across platforms.
3. A chain ABI specified jointly with the chain team, implemented by the chain, and exercised by Clarity plus one other source-language prototype.
4. A static cost analyzer that emits a sound worst-case fuel bound for every public function of any well-typed contract.
5. A source-verification service or tool that rebuilds and verifies any deployed Clarity module from its source.

Architecture: Clarity on a Wasm Chain

This document describes how Clarity source becomes code running on the new chain, what the chain guarantees for every module, and where the compiler sits in the trust path.

1. Shape

There are three parts, and only the middle one is shared between languages:



- The **toolchain** is Clarity-specific and runs off chain. The chain never sees Clarity source.
- The **chain ABI** is the contract between any compiler and the chain. It is specified jointly with the chain and shared by every source language.
- The **chain runtime** executes Wasm and enforces the guarantees in section 5 for every module.

2. The Clarity toolchain

Illustrative crate layout; names are placeholders.

```
clarity-syntax      lexer, parser, source spans
clarity-types       type checker, overload resolution, instantiation (spec ch. 2)
clarity-std         the prelude source (spec/std/prelude.clar) and its tests
clarity-compiler    inline expansion, lowering to Wasm, prelude linking,
                    interface metadata, provenance section
clarity-analyzer    static cost bounds, asset-flow and authorization analyses
clarity-verify      rebuilds a module from source and compares it
```

Rules that bind every Clarity compiler:

- **Deterministic output.** The same build inputs produce a byte-identical module on every platform. No timestamps, paths, hash-map iteration order, or host-dependent floating point in the output.
- **Build inputs are explicit.** Compilation is not a function of source alone. Type-checking a `contract-call?` with a literal target reads the callee's deployed interface (spec 2.5.8), and the

singleton-contract subtyping rule reads it too (spec 2.3.2). The inputs are therefore the source, the compiler version, the interface of every contract referenced by literal, and the deploying principal, because deployer-relative `.name` references are resolved at compile time. A module is therefore specific to its deployer, and the chain has no relocation step and no Clarity-specific name resolution. Deployed interfaces never change, so recording which ones were used is enough to reproduce the build.

- **Provenance.** Each module carries a custom section with the compiler identifier and version, the language version, a hash of the source, and the hash of each callee interface the build read, keyed by contract principal. `clarity-verify` fetches those interfaces from the chain and checks the hashes before rebuilding.
- **The prelude is linked into each module.** Only the library instances a contract uses are included, compiled like user code. There is no chain-provided library module, so the chain carries no Clarity code. Consequence: a faster library implementation ships in a new compiler release. Contracts already deployed keep the code they were deployed with, and no chain upgrade is needed.
- **Hashes are compiled, not host functions.** `sha256`, `sha512`, `sha512/256`, `keccak256`, and `hash160` are Wasm compiled from their Rust implementations and linked into the module like the prelude. Signature schemes stay host functions (section 4).
- **A new language version is a compiler release.** It needs the chain only when it depends on a new host function or ABI feature.

3. The module format

A deployed contract is one Wasm module plus its interface:

- **Exports**: one function per public and read-only function, and an initializer the chain runs once at deploy (Clarity's top-level expressions and `define-constant` / `define-data-var` initializers).
- **Imports**: host functions only (section 4). A module that imports anything else is rejected.
- **Interface metadata** (a custom section, validated by the chain): each exported function's name, parameter and result types in ABI types, and read-only mark; the traits the contract defines and claims to implement; its token definitions. This is what other contracts compile and dispatch against. The ABI type grammar includes an empty type (`never` in Clarity, spec 2.3.3) so that a function can declare that it never returns `err`; boundary validation admits no value for it.
- **Provenance** (section 2).

At deploy the chain checks that the module is valid Wasm within size limits, imports only host functions, exports exactly the functions its interface declares with matching Wasm signatures, and that every implemented-trait claim holds against the interface. It then runs the initializer.

Values across calls

Values cross a contract call in the ABI's canonical encoding (the value model and wire format, formalizing SIP-005). The chain **decodes and validates every argument against the callee's declared parameter types, and every result against the declared result type**, before the other side sees it. For a trait-typed parameter, the chain checks that the principal's interface implements the trait.

The chain cannot trust any caller's compiler, so this validation is what lets a Clarity contract rely on its own types when it is called from a module written in another language.

4. The host interface

Everything a module can do beyond pure computation goes through host functions. The Clarity primitives in spec/03-primitives.md that pass the layering rule's host-access test map onto them.

Group	Host functions
Storage	data-var get/set, map get/set/delete, keyed by the calling contract
Assets	fungible and non-fungible token mint/burn/transfer/balance/owner/supply, STX transfer/burn/account; every movement recorded in the transaction's asset map
Context	tx-sender, contract-caller, sponsor, current contract, block heights and time, chain id, network flags, liquid supply
Chain data	Stacks block, tenure, and burn block lookups; Bitcoin transaction output parsing. All total: they return `none` rather than fail
Calls	contract-call by principal and function name, with argument and result validation (section 3)
Frames	`as-contract?` and `restrict-assets?` frames: context switch, nested rollback, allowance checks
Events	print and asset events
Cryptography	signature schemes (secp256k1 recover/verify/decompress, secp256r1, ed25519). Hashes are compiled into the module (section 2)

The allowance comparison for `as-contract?` and `restrict-assets?` is host code, not compiled code over a host-provided movement log, so the guarantee holds for every module whatever its source language.

Each host function has a fuel charge, and storage operations are also charged in read/write count and length dimensions. The fuel schedule and charge table are part of the chain ABI. They may change at epoch boundaries, and a change applies to every deployed module: a module's cost depends only on its code and the engine's schedule, not on its source. Cost bounds are therefore stated per schedule version.

5. Where each guarantee lives

Guarantee	Enforced by	Holds for
Execution stops within the fuel budget	Chain (fuel metering)	Every module
No reentrancy; bounded call depth	Chain (call dispatch)	Every module

Guarantee	Enforced by	Holds for
No writes inside a read-only call	Chain (host functions reject writes in a read-only frame)	Every module
A public call that returns `err` has its state changes discarded	Chain (frame commit/discard on the result)	Every module
Values at call boundaries match the declared interface	Chain (section 3)	Every call
Post-conditions; `as-contract?` / `restrict-assets?` allowances	Chain (asset map, frames)	Every module
Termination by construction; worst-case fuel of the module's own code computable in advance	Clarity language, preserved by the compiler	Clarity modules
Worst-case fuel of a whole call graph computable in advance	Static cost analyzer over verified source	Call graphs whose every callee is verified Clarity, reached by literal target. A trait call or a call into another language is bounded only by the remaining budget, and the analyzer reports it as such
Complete static analysis; tractable proofs	Clarity language	Clarity modules with verified source
No undefined behavior; responses must be handled	Clarity type checker and compiler	Clarity modules

The first six rows are why a Clarity contract can call a module from another language safely. The last four are what Clarity adds over any other language on the chain.

6. The trust path

The compiler is in the trust path: a compiler bug can make the deployed code differ from the audited source. The defenses:

1. **Spec and conformance suite.** A correct compiler is defined by the spec, not by an implementation. Every compiler release passes the suite.
2. **Differential testing and fuzzing against a direct evaluator.** The direct evaluator is an interpreter that runs Clarity source by applying the spec's dynamic-semantics rules one at a time, with no lowering, no Wasm, and no optimization. It is a much smaller program than the compiler and a different kind of program, so the two are unlikely to share a bug. Differential testing runs the same contract and the same calls through both and compares every observable result: return values, events, asset movements, and aborts. Fuzzing supplies those contracts and calls by generating them, well-typed, in volume. Any disagreement is a bug in the compiler, in the evaluator, or in the

spec, and the spec adjudicates. The evaluator is only a test oracle: it never runs on chain, is never a fallback execution path, and has no authority over the compiler. A second production implementation would make "whatever the implementation does" the de facto spec again, which is the legacy situation this design leaves behind.

3. **Reproducible builds and source verification.** Source is published in an off-chain registry keyed by the source hash in the module's provenance section; the chain stores only the module. `clarity-verify` rebuilds a module from its claimed source and compiler version and compares bytes. Wallets and explorers show whether a module's source is verified.
4. **Analyses attach to the triple.** A cost bound or proof is stated for a (source hash, compiler version, module hash) triple and is meaningful for a deployed module only when that triple verifies. A cost bound also names the fuel-schedule version it was computed for (section 4). Bounds need both halves of the triple: loop and size bounds come from the verified source, because only Clarity source guarantees bounded loops, and fuel comes from the module, because that is what the chain meters. The chain sees only the module, so the ABI carries no bound certificates.

7. Legacy contracts

Clarity 1–6 contracts keep running on the interpreter, and the interpreter and the Wasm engine can call each other. How that works is a chain concern outside the scope of these documents.

It is out of scope, not unimportant. Tokens, PoX, and sBTC are legacy contracts, and a new contract that cannot call them has no ecosystem. The chain owns the mapping from a legacy contract's analysis to an ABI interface, and the mapping is not one to one. Known mismatches the chain must decide, with input from the language:

- **Tuples.** Legacy admission lets a tuple with extra fields pass where a smaller one is declared, so a legacy function can return a value with fields its interface does not name. vNEXT requires exact field sets (spec 2.3.2). Boundary validation must either strip extra fields on the way out of legacy code or reject them.
- **List union types and trait values.** Legacy lists may carry a union of trait types and contract literals, and legacy trait values carry static type information at runtime. The ABI value model has neither; a trait value is a principal (spec 2.7.1).
- **Read-only marks.** A vNEXT read-only function may call a legacy function only if the legacy interface marks it read-only, so the mapping must carry that mark.
- **Error-code and field-name conventions** (`error_code`, `stx-transfer-memo?`) are the legacy contract's and are not translated.

8. Open questions

1. **Wasm memory limit.** The language's intermediate value size limit (spec 2.8, provisionally 8 MiB) must stay below the memory a module may use, and exceeding that memory is a runtime error (spec 2.7.2). The chain ABI must fix the limit, per transaction or per call frame.

2. **Module size limit.** It is the only limit on a contract's size after inline expansion and instantiation (spec 2.8), and the deploy check in section 3 enforces it. The chain ABI must fix the value.

Specification Plan

Scope decision: two documents, one owner each

- **The Clarity specification** defines the language: syntax, types, semantics, the standard library, and what a correct compiler to Wasm produces. It is owned by the language project. Drafts live in spec/.
- **The chain ABI** defines what any module can rely on: the module format, interface metadata, value encoding, host functions and their charges, call semantics, runtime guarantees, and the fuel schedule. It is owned jointly with the chain and shared by every source language. The Clarity spec refers to it and never restates it.

Clarity 1–6 are out of scope. They remain defined by their implementation on the legacy track. The new language owes them nothing in syntax or semantics, and the disposition table in spec/04-standard-library.md serves as the porting guide.

The language is designed **spec-first**: the spec is written and reviewed before the reference compiler, and the compiler is validated against it, never the other way around.

Structure decision: minimal core plus a standard library

The spec is organized as **a small core language and a standard library written in that language**.

- The **core** is the syntax, the type system, about a dozen special forms (binding, `match`, `return`, definitions, calls, context scoping), and about eighty primitive operations. A construct is core if it needs the host, cannot be expressed in the language, is a cryptographic standard, or is a base operation on a type's representation (spec/README.md states the rule). Cost is not a test.
- The **standard library** is everything else: control flow (`if`, `unwrap!`, `try!`, `asserts!`, `default-to`), predicates, derived arithmetic (`mod`, `pow`, `sqrti`), sequence helpers (`append`, `index-of?`, `replace-at?`), integer/string/byte conversions, `verify-merkle-proof`, `map-insert?`, and more, about forty definitions. It is ordinary Clarity source (spec/std/prelude.clar), versioned and frozen with the language, compiled and linked into each module like user code, and it has no powers a contract author lacks.
- **Cost is Wasm fuel, not a language concern.** Contracts are metered by the fuel their Wasm consumes, plus the chain's charges for host functions. The language has no cost table, so the "cost table disagrees with behavior" bug class cannot occur. The prelude's source defines results; a later compiler may link a faster implementation, which changes nothing for contracts already deployed.

Spec structure

A versioned document set, one complete spec per language version (the ECMAScript/Rust-edition model; no delta chains to reconstruct):

1. **Design principles**: decidability as an invariant; every construct must preserve statically bounded execution. This chapter is the rejection criterion for future proposals.
2. **Lexical structure and grammar**: complete EBNF for the surface syntax, including the honest edge cases (string escapes, integer literal bounds, principal literals, identifier rules).
3. **Type system**: types, subtyping, trait typing (fully static and decidable), inference rules written formally enough to derive a checker from. Every rule numbered and conformance-tested.
4. **Static semantics**: the analysis passes the language requires: read-only checking, trait conformance, definition ordering. Specified as judgments, not as descriptions of a checker.
5. **Dynamic semantics**: evaluation order, error, abort, and rollback semantics, context scoping (`as-contract?`, `restrict-assets?`), storage visibility, event emission. Written in a proof-friendly style (small-step or big-step operational semantics) so verification tools can consume it. This is the semantics every compiler must preserve.
6. **Compilation to Wasm**: the module layout Clarity uses within the chain ABI, the lowering of each core form and primitive, prelude linking, the interface and provenance sections, and the determinism rules. It also carries the argument that loop bounds guaranteed by the source types survive compilation, which is what makes "worst-case fuel of any function is statically boundable" a theorem. The static cost analyzer is derived from that argument.
7. **Primitives and standard library**: every core primitive with its signature, semantics, and error cases (spec/03-primitives.md); the standard library, specified by its source (spec/std/prelude.clar), plus the Clarity 1–6 disposition table (spec/04-standard-library.md).

The chain ABI is a separate document with its own chapters: value model and wire format, module format and interface metadata, host functions and their charges, call dispatch and boundary validation, runtime guarantees, and the fuel schedule.

The conformance suite

The suite is what makes the spec real, and it is the highest-leverage artifact of the effort. Without it the spec is a description that implementations resemble to varying degrees; with it the spec is a contract that an implementation either meets or does not.

What a vector is

A **conformance vector** is one test case written as data rather than as code: an input, the outcome the spec requires for that input, and the identifiers of the normative rules it exercises. Because a vector is data, every implementation runs the same vectors through its own runner, so the suite never depends on any one implementation's language or test framework. When a vector fails, it names the rule that was broken.

An illustrative language vector (the concrete format is Phase 0 work in 05-roadmap.md, and rule identifiers are not yet assigned):

```
id:     lang/arith/add-uint-overflow
rules:  03 §3.1 (+ overflows), 02 §2.7.2 (runtime errors abort)
source: (define-public (f)
          (ok (+ u340282366920938463463374607431768211455 u1)))
call:   f
expect: abort, class arithmetic-overflow
```

Three kinds of vector

Kind	Input	Expected outcome	Who must pass
Language	Contract source, plus calls	A compile error of a specified class, or for each call: return value, events, asset movements, and whether it aborts	Every Clarity compiler
ABI	A module, plus chain actions	Deploy and boundary validation, rollback, reentrancy and read-only rejection, host function results and charges	Every chain implementation, independent of source language
Golden build	Source and a compiler version	The exact module bytes	That compiler release, on every supported platform

Language and ABI vectors are the suite proper: they pin behavior the specification requires, and they are written with the spec and versioned with it. Golden build vectors are different in kind. They pin nothing about the language; they pin a particular compiler release, so that reproducibility (section 6 of 03-architecture.md) is testable rather than asserted. Each compiler release publishes its own.

What a vector deliberately does not pin

The suite checks what the spec requires and nothing more, so that implementations keep room to improve:

- **Fuel.** A language vector never states how much fuel a call consumes. A better compiler is allowed to produce cheaper code, and a vector that pinned fuel would forbid it. Charges for host functions are pinned by ABI vectors, because the ABI's fuel schedule is normative for the chain.
- **Module bytes.** A language vector never states what Wasm a compiler emits, only how the result behaves. Byte-exactness is the job of golden build vectors, and only for one release at a time.
- **Error text.** A compile-error vector names the class of error (an arity mismatch, an undefined name, a read-only violation), not a message. Messages are a tool's own concern.

How rules and vectors bind each other

Principle P8 (spec/01-principles.md) states the contract: every normative rule cites the vectors that exercise it, and no rule is frozen without vectors. The runner enforces this both ways. A rule that cites no

vector is unfrozen and shows up in a coverage report. A vector that cites no rule is testing something the spec does not say, which is a spec bug to fix before the vector is accepted.

Which side is the oracle depends on where the rule lives:

- For the core language, the prose rule is normative. If a compiler and a vector disagree, one of them has a bug, and the disagreement is adjudicated explicitly and recorded (P8).
- For the standard library, the prelude source is normative (spec/04-standard-library.md). Library vectors are derived from the source, and if a vector and the source disagree, the vector is wrong. The same vectors are what hold a hand-written or better-compiled replacement, linked by a later compiler in place of the source, to identical results, events, and errors.
- For the chain, the ABI document is normative, and ABI vectors are its suite.

Where the suite sits among the other defenses

The suite is the floor, not the ceiling. It is a fixed set of cases, so it catches regressions and divergence on behavior someone thought to write down. Differential testing against the direct evaluator and fuzzing (section 6 of 03-architecture.md) search above that floor for behavior nobody wrote down. Every bug they find becomes a vector, so the floor only rises.

Language design opportunities

Candidates for the RFC process, listed to scope ambition, not to commit:

- Consistent naming and calling conventions across the standard library (the `unwrap!`/`unwrap-panic`/`try!` family, `?`/`!` suffix discipline).
- A trait system that is fully static, decidable, and pleasant, designed for boundary validation by the chain from the start.
- Cost transparency in the language: contracts and functions can declare fuel bounds that the analyzer verifies; callers can rely on them.
- Richer (still bounded) iteration and data manipulation, so common patterns stop requiring fold contortions. Every addition passes the chapter-1 decidability test.
- First-class annotations for verification (invariants, pre/postconditions) consumed by the proof toolchain.
- Remove the footguns of Clarity 1–6.

Process and governance

- Spec development runs as an open RFC process in the language repo. Each chapter and each language feature is an RFC with a decidability analysis as a required section.
- A new language version is a spec release plus a compiler release. It needs the chain only when it depends on a new host function or ABI feature, and those changes follow the chain's own governance.

- The spec versions with the language: each version gets its own complete spec. The chain ABI versions separately and is additive once frozen.

Findings from drafting the prelude

Writing the prelude is the test of the core design. Findings so far:

- An earlier draft returned the conversion, search, and Merkle functions to the core because their library versions cost more than a native under a per-operation cost table. With fuel as the metric and optimization done by a compiler linking faster code, that argument is gone, and they are library again. The core is smaller and there is no per-construct cost table.
- Template checking of generic bodies carries more weight than planned. It lets `index-of?` and the string parsers be one definition across sequence kinds instead of four, at the price of errors reported inside the prelude (spec 2.6.2).
- Per-kind constants (digit tables, the minus sign) need small overloaded helpers, because a generic body cannot write a literal of an unknown kind.
- `replace-at?` exposed a gap in the size rules: an intermediate `concat` has a larger bound than any value it can hold. Intermediate expressions are checked against a separate, larger limit (spec 2.3.1).
- Rest parameters are strict: the extra arguments are built into a list before the body runs. That is right for `max`, `+`, and `concat`, but it means n-ary short-circuiting cannot be written in the language, so `and` and `or` moved from the library into the core (spec 2.5.3).
- `never` as the type of `(return ...)` and `(panic)` makes every early-exit form typeable with ordinary lub rules, with no special cases.
- Inline functions cannot be passed to `filter`, which forces predicates to be private functions (spec 2.6.3).
- Folds whose initial value has a smaller type than the step's result (`std-reverse-16` starts from `0x`, `std-render-decimal` from `""`, `index-of?` from `none`) rely on instantiation finding the accumulator type by iteration (spec 2.6.2). Each closes in two steps; a fold that needs a wider start states it with `as` (2.5.10).
- Overloading and static reference kinds are each justified by a small group of prelude functions (`unwrap!`/`try!` and the integer families; `map-insert?`).

Roadmap

Phased plan for the new Clarity on the Wasm chain. Phases overlap where the dependency summary allows. The legacy track (running Clarity 1–6 on the interpreter) proceeds separately and is not covered here.

Phase 0 — Foundations

- Stand up the language repository and RFC process.
- Carry the working draft in spec/ into RFCs: principles, core language, primitives, standard library.
- Open the chain ABI draft jointly with the chain team: value model, module format, host function list, call semantics.
- Define the conformance-vector formats (language, ABI, golden build) and build the vector runner early.

Phase 1 — Spec and ABI

- Settle the open questions in each spec chapter; freeze the core forms and primitive list.
- Write chapter 6 (compilation to Wasm) alongside the ABI, so the lowering and the host interface are designed against each other.
- Write language vectors as each chapter lands, not after.
- Reach ABI v0: enough to deploy and call a module, store data, move assets, and call another contract.

Phase 2 — Reference compiler

- Front end: parser, type checker, overload resolution, instantiation, inline expansion.
- Back end: lowering to Wasm, prelude linking, interface and provenance sections.
- Determinism from the first commit: golden build vectors in CI on every supported platform.
- A direct evaluator: an interpreter that runs Clarity source by applying the spec's semantic rules one at a time, with no Wasm in the path. It exists only as a test oracle. Differential testing runs the same contracts and calls through the evaluator and through the compiler and compares the results, and fuzzing generates those contracts and calls in volume (03-architecture.md section 6). The evaluator never runs on chain.
- Exit criterion: the compiler passes the language vectors.

Phase 3 — Chain integration

- Chain devnet executing Clarity-compiled modules through ABI v0.

- Host functions, fuel metering, and every runtime guarantee in 03-architecture.md section 5, each covered by ABI vectors.
- A second source-language prototype (a small subset of Rust or AssemblyScript is enough) deploying against the same ABI, to catch Clarity-shaped assumptions before the ABI is frozen.

Phase 4 — Trust path

- `clarity-verify`: rebuild a module from source and compiler version and compare bytes.
- A source registry keyed by the provenance hash, and verified-source display in at least one wallet and explorer.
- Compiler hardening: sustained fuzzing, differential testing against the oracle, external audit.

Phase 5 — Cash in decidability

Flagship deliverables, in value order:

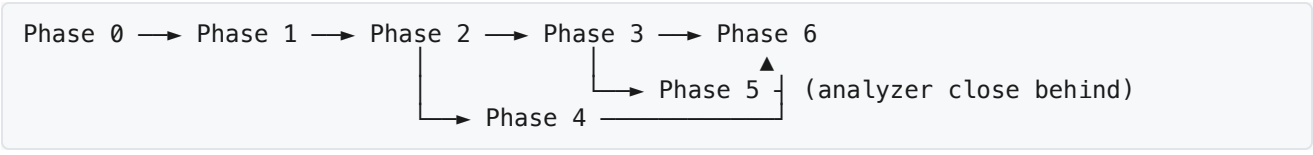
1. **Static cost analyzer**: sound worst-case fuel bounds per public function, from source, the verified module, and the chain's host-charge table. Surfaces: pre-deploy reports, wallet max-fee display, fee estimation without simulation, block packing.
2. **Contract property checker**: SMT-backed verification of standard invariants (supply conservation, authorization, no fund lockup) using the spec's verification annotations; a path toward machine-checked contracts as a normal deployment artifact.
3. **Whole-ecosystem analysis**: precise asset-flow and interaction-graph tooling over verified source.

These are ongoing products, not a phase that completes. Item 1 should land close behind launch, because it is the concrete answer to "why a dedicated language".

Phase 6 — Launch

- Freeze the language spec for the first version and ABI 1.0.
- Publish the porting guide from Clarity 1–6, built on the disposition table in spec/04-standard-library.md.

Dependency summary



Phase 4 needs only the compiler. Phase 5 needs the chain's fuel schedule and host-charge table from Phase 3, so the analyzer's design can start with Phase 2 but its bounds cannot be stated until ABI v0

fixes the schedule.

Top risks

1. **ABI frozen too early**: a wrong surface forever, for every language. Gate the freeze on Clarity plus the second-language prototype both shipping against it.
2. **Compiler bugs**: deployed code differs from audited source. Conformance vectors, differential testing against the oracle, fuzzing, reproducible builds, and an external audit before launch.
3. **Spec drifts from the compiler**: the ECMAScript 4 failure mode. Vectors are written with each chapter, and the compiler implements the spec, never the reverse silently.
4. **Static bounds lost in compilation**: if the lowering obscures loop bounds, the analyzer cannot prove them from the module. The analyzer therefore takes loop bounds from verified source and fuel from the module, and chapter 6 must carry the preservation argument that the module stays within the bound its source implies.
5. **Justification pressure**: if Phase 5 slips indefinitely, "just use Rust on the Wasm chain" wins by default. Treat the static cost analyzer as a headline deliverable with the same priority as the compiler.
6. **Fuel schedule churn**: schedule changes happen at epoch boundaries and re-price every deployed contract, so a published bound holds only for the schedule version it names. Agree with the chain team on how often the schedule may change before launch.
7. **Bounds stop at foreign code**: a `contract-call?` through a trait, or into a module from another language, has no static bound, and most real contracts make such calls. If the analyzer can only bound leaf contracts, the max-fee product does not ship. Mitigations: bounds compose over verified-Clarity call graphs; the analyzer reports the remaining budget at each unbounded call rather than refusing; and the language design opportunity in 04-spec-plan.md (declared, checked fuel bounds that callers can rely on) is the long-term answer for trait calls.

Issue Tracker Disposition

Every open issue labeled `clarity` in stacks-core (as of 2026-09-09), with what the specification draft in spec/ does about it. Issues are grouped by disposition. "Addressed" means the draft's rules make the reported problem impossible or the request part of the language; the referenced section is where. Section numbers refer to the specification chapters in spec/.

Addressed by the draft

Issue	Summary	Where
#7312 (and #7147)	Integer to fixed-width bytes, abort on overflow	`uint-to-buff-be/le`, `int-to-buff-be/le` primitives, 3.4
#6831	A trait defined in the contract cannot be used at the contract's top level	2.4.2: local definitions are fully usable in top-level expressions
#6485	`concretize` mishandles lists mixing trait values and contract literals	No `ListUnionType`. A contract literal has a singleton type that is a subtype of every trait it implements, and list literals take the lub (2.3.2, 2.5.6)
#5128	`get-block-info? miner-address u0` makes the transaction unmineable	3.9: chain-data lookups are total and return `none`
#4751	`(is-eq trait-value .contract)` passes the checker and fails at runtime	`(trait X) <: principal`, so the lub is a principal and the comparison is well-typed; runtime values are plain principals (2.3.2, 2.7.1)
#4622	Checker rejects a list of distinct contract literals	Lub of singleton contract types is `principal` (2.3.2)
#4431	Duplicate parameter names and wrong argument counts accepted	2.4.1: parameter names distinct; 2.6.2 and 2.6.5: declared arity, checked statically
#3566	Cannot call a trait extracted from a tuple	2.5.8: any trait-typed expression is a call target
#3176	Binding a reserved keyword passes analysis, fails at runtime	2.5.2: no shadowing of any in-scope name, checked statically. The forward-compatibility concern raised in the thread is moot under frozen language versions: a new keyword only exists in a new version, and old contracts keep theirs
#1981	Read-only functions cannot call trait functions	2.4.1 and 2.7.4: traits may mark functions `read-only`; implementations must be read-only; read-only callers may call them through the trait

Issue	Summary	Where
#1638	Constants cannot be used where literals are required	2.4.1: static constants (built from literals) are usable as size terms and `contract-call?` targets
#1171	Type aliases	`define-type`, 2.3.5
#3056 (referenced by #7312)	Variadic `concat`	`concat` has a rest parameter (3.3); rest parameters are a user feature (2.6.5)

Candidate additions

Feature requests that fit the language's positioning but are not yet in the draft because each deserves a short RFC (layer, host interface impact, exact signature).

Issue	Request	Draft position
#3568	Schnorr (BIP-340) signature verification over secp256k1	Strong fit for a Bitcoin-native language (Taproot, FROST). Proposed: `secp256k1-schnorr-verify : ((buff 32) (buff 64) (buff 32)) -> bool`, `false` on malformed input, placed in 3.6. Recommend including in vNEXT
#3849	`tx-fee` context value	A `fee` field of `(get-current-context)` (3.9); requires the host interface to expose the fee of the executing transaction. Recommend including; decide the value for `contract-call?` frames of sponsored transactions
#3975	256-bit integer arithmetic for Schnorr, ZK, and curve math	RFC. A `(buff 32)`-based `wide-add`/`wide-mul`/`wide-mod` family is expressible in the language, so under the layering rule it is library code, with a Wasm replacement later if it is hot. A new 256-bit integer type would instead be a core change. The RFC chooses between the two. If #3568 ships as a primitive, the most pressing motivation goes away
#6320	`code-body-of?` / per-definition code hashes	`contract-hash?` is in the draft (3.5). The thread converges on per-top-level-definition hashes (`has-definition-value?` or a MAST) as the safe version of introspection; returning source text invites contracts to parse Clarity in Clarity. The chain holds only the module, so an on-chain per-definition hash would be over compiled code and would depend on the compiler. The RFC decides between that and per-definition source hashes carried in the interface metadata

Out of scope for the language spec

Issue	Why
#4487, #4288	Signer-voting and StackerDB boot-contract API design
#3977, #3878, #3877	sBTC and PoX contract migration checklists (2023)
#4030	Boot-contract test infrastructure

Issue	Why
#3350, #3302	Diagnostic quality. Implementation concerns for the reference compiler, though #3350's root cause (a `uint` expression where a literal is required) is removed by static constants

Patterns worth noting

Three themes recur across the tracker and shaped rules in chapter 2:

- **Checker and runtime disagree** (#4751, #4431, #3176, #6485, fixlist items 5 and 10). The draft's answer is structural: one lookup order, no shadowing, declared arity, and a value model with no static type information at runtime, so there is nothing for the two to disagree about.
- **Literals where expressions should do** (#1638, #3350, #6831). Static constants and the "any trait-typed expression" rule remove the special cases.
- **Traits are second-class** (#3566, #1981, #6485, #4622, #4751). Making `(trait X)` an ordinary type with a subtyping relation to `principal` resolves every one of these without a dedicated rule.

Clarity vNEXT Specification (working draft)

This directory holds the working draft of the specification for the new Clarity language (working name **vNEXT**), which compiles to Wasm for the new chain. It follows the scope decision in ../04-spec-plan.md: Clarity 1–6 are not specified here, and the chain ABI that every source language shares is a separate document that this spec refers to.

> [!IMPORTANT] Everything in this directory is a draft intended to seed the RFC process. Nothing here is normative yet. A chapter with open questions lists them at the end; an open question tagged `RFC` needs its own design RFC before the chapter can be frozen.

The two-layer design

vNEXT is specified as **a small core language plus a standard library written in that core language**. This is the single most important structural decision in the spec, and every chapter is organized around it:

Layer	Contents	Where specified
Core language	Syntax, types, a fixed set of *special forms* (binding, branching, early exit, definitions, calls, context scoping) and a fixed set of *primitive operations* (host access, cryptography, and the base operations on each type's representation)	02-core-language.md, 03-primitives.md
Standard library	Every other construct a contract author uses: `if`, `unwrap!`, `try!`, `asserts!`, `default-to`, `mod`, `pow`, `append`, `index-of?`, `map-insert?`, integer/string conversions, `verify-merkle-proof`, and so on. Written in vNEXT and linked into every module by the compiler	04-standard-library.md, std/prelude.clar

The layering rule

A construct belongs in the core **only if** it passes at least one of these tests; otherwise it belongs in the standard library:

1. **Host access.** It reads or writes contract state, moves assets, reads chain data, reads the transaction context, emits events, or calls another contract. (Examples: `map-get?`, `ft-transfer?`, `get-current-context`, `print`, `contract-call?`.)
2. **Not expressible.** It cannot be written in the language at all because it introduces binding, branching, early exit, definitions, generics, or type-directed behavior, takes a type or size argument, or needs a function argument that the language cannot close over. (Examples: `let`, `match`, `return`, `define-map`, `is-eq`, `as-max-len?`, `from-consensus-buff?`, `uint-to-buff-be`, `map`, `filter`.)
3. **Cryptographic standard.** It is a hash function or signature scheme defined by an external standard. These are implemented once, by audited host libraries, not reimplemented in the

language. Constructions built from them are library code. (Examples: `sha256`, `secp256k1-verify`; but `verify-merkle-proof` and `principal-of?` are library.)

4. **Representation base.** It is one of the minimal operations on a built-in type's representation that everything else is written in terms of: integer `+ - * /`, bitwise `and` / `or` / `xor` and shifts, comparison, one conversion per pair of representations (`to-int`, `to-uint`, `buff-to-uint-be`), and sequence length, concatenation, indexing, and slicing. Anything derivable from these is library. (`mod`, `pow`, `sqrti`, `buff-to-int-le`, and `int-to-ascii` are library. The `*-to-buff-*` family is core by test 2, not this one: each takes a size argument.)

Cost is not a test. Contracts are metered by the fuel their compiled Wasm consumes, so a library function costs what its code costs to run, and a function that turns out to be hot is optimized by a compiler that links a faster implementation, not by moving it into the core (chapter 4.1).

The standard library has **no special powers**. It is ordinary vNEXT source that uses only features available to every contract. If the library needs a feature (generics, inline functions, overloading, static references), that feature is a user feature and is specified as such.

Why this shape

- **The spec shrinks where it matters.** The legacy special forms with bespoke lazy-evaluation and typing rules (`if`, `unwrap!`, `try!`, `asserts!`, `default-to`, ...) and the derived arithmetic, sequence, and conversion natives become library definitions whose semantics are *defined by their source*. There is nothing to specify about `unwrap!` beyond the four lines that implement it. The remaining natives become about 80 primitives with uniform typing rules.
- **The language has no cost table.** Execution is metered in Wasm fuel, and the only other charges are the chain ABI's charges for host functions. There is no per-construct cost table to keep in sync with behavior, and nothing about cost to specify for the library.
- **Compilers cannot disagree about the library.** Any compiler that implements the core correctly gets the library for free.
- **Design pressure goes the right way.** Each proposed addition must either pass the layering rule (and pay the full price of a core change) or be written as library code, where it is cheap, reviewable, and provably bounded.

Chapters

Chapter	Status	Maps to plan chapter
01-principles.md	Draft	1 (design principles)
02-core-language.md	Draft	2, 3, 4, 5 (grammar, types, static and dynamic semantics)
03-primitives.md	Draft	7 (native reference, reduced to primitives)
04-standard-library.md	Draft	7 (library), plus the legacy-to-vNEXT disposition table

Chapter	Status	Maps to plan chapter
std/prelude.clar	Draft	The library itself
Compilation to Wasm	Not started	6 (lowering, prelude linking, determinism, bound preservation)
Chain ABI	Not started; a separate document shared by every source language	ABI (value encoding, module format, host functions, fuel schedule)
Conformance vectors	Not started	Conformance suite

Conventions

- **MUST**, **MUST NOT**, **SHOULD**, **MAY** carry their RFC 2119 meanings.
- Type signatures use the notation `name : (arg-types) -> result-type`. Uppercase single letters (`T`, `E`, `K`, `V`) are type variables; `N`, `M` are size (length-bound) variables. A signature written "for `I` in {`int`, `uint`}" is a family of signatures, one per listed type.
- Source examples are written in vNEXT syntax, which differs from Clarity 1–6 where noted.
- "Legacy" means Clarity 1–6. It appears only in comparisons and in the porting table of chapter 4.
- "The chain" means any chain implementation of the chain ABI, and "host function" means a function the chain provides to modules through it.

1. Design Principles

These principles are the rejection criteria for every construct in vNEXT, present and future. A proposal that violates one of them is out of scope regardless of its merit on other axes.

P1. Decidability is the invariant

Every vNEXT program terminates, and the worst-case cost of every function is a computable function of its parameter types. Concretely, the language MUST keep all of the following true:

- Every type has a statically known maximum size. Lists, buffers, and strings carry length bounds. There are no unbounded data structures.
- There is no recursion, direct or mutual, and no unbounded loop. Iteration exists only over sequences whose length is bounded by their type.
- Inline expansion and generic instantiation terminate, because the definition graph is acyclic.
- Call depth is bounded, and a contract cannot be re-entered while a call into it is active.

Any construct that cannot be shown to preserve these properties is rejected. This is the property that makes exact static cost bounds (in fuel, P4), complete static analysis, and tractable proofs possible (../01-why-clarity.md).

P2. Minimal core

The core language contains only what passes the layering rule in README.md: host access, constructs that cannot be expressed in the language, hash functions and signature schemes defined by external standards, and the base operations on each type's representation. Everything else is standard library.

The core is therefore small enough to hold in one head: about a dozen top-level definition forms, about a dozen expression forms, and about eighty primitive operations with uniform typing rules. The library holds everything else a contract author uses, about forty definitions.

P3. The standard library has no special powers

The standard library is ordinary vNEXT source. It uses only features that every contract can use, it is type-checked by the same rules, and it is compiled and metered like any other code. Anything the library can do, a contract author can do. If the library needs a language feature, that feature is designed and specified as a user feature.

Corollary: a proposal to "just add a native" for something expressible in the language is rejected. The right fix is a library function, or if the library cannot express it well, a general language feature that lets it. Speed is not a reason to add a native; see P4.

P4. Cost is not a language concern

The language assigns no cost to any construct. Execution is metered in the fuel the chain's Wasm engine consumes running a contract's module, plus the chain ABI's charges for host functions (storage I/O, asset operations, chain-data lookups, contract calls, cryptography). Both belong to the chain ABI, not to this specification.

Consequences:

- There is no per-construct cost table, so there is no class of bugs where a cost table disagrees with behavior, and no cost chapter that grows with the language.
- A library function costs the fuel of its compiled code, like user code.
- The static bound still exists. By P1, every loop and every value is bounded by its type, so the worst-case fuel of every function is computable from the source, its compiled module, and the chain's charge table (chapter 6 of the plan carries the argument that compilation preserves the bounds).

P5. The deployed Wasm is law; the source is its specification

The chain stores and executes Wasm modules and never sees Clarity source. What a deployed contract does is what its module does.

This specification defines what a module compiled from a given source MUST do: its results, state changes, events, and aborts. A compiler that produces a module whose behavior differs from the source's meaning has a bug. The language keeps its guarantees on chain only through compilers that meet this spec, which is why compilers are held to the conformance suite and to deterministic, verifiable builds (../03-architecture.md).

The standard library is versioned with the language and frozen when the language version is frozen; its source defines its results. The compiler links the library code a contract uses into that contract's module. A later compiler MAY link a faster implementation, which MUST produce identical results; contracts already deployed keep the code they were deployed with. A contract never observes a library change.

P6. One way to do each thing

- No aliases. Legacy pairs such as `element-at` / `element-at?`, `index-of` / `index-of?`, and `xor` / `bit-xor` collapse to one name.
- No overlapping forms. `as-contract` is gone; `as-contract?` remains. `stx-transfer-memo?` is folded into `stx-transfer?`. `(tuple ...)` is gone; `{...}` remains. `begin` is sugar for `let` with no bindings.
- Naming discipline, applied uniformly to core and library:
- `name?` returns an `optional` or a `response`.
- `name!` may return early from the enclosing function.

- `is-name` is a predicate returning `bool`.
- Names are lowercase words separated by `-`. Tuple fields follow the same rule (`error-code`, not `error_code`).
- Declared arity. Every call's arity is fixed by its definition: exactly the declared parameters, or, for a definition ending in a rest parameter `(name T...)`, the fixed parameters plus one or more arguments collected into a list (2.6.5). `(+ a b c)` works because `+` has a rest parameter, not because `+` is special; a user function can have one the same way. There are no optional parameters and no per-arity typing rules.

P7. No undefined behavior, no silent failure

- Arithmetic overflow, division by zero, out-of-range conversions, and invalid shift amounts abort the transaction deterministically.
- Errors are values (`response`, `optional`) or aborts. There is no third kind of outcome, and a `response` cannot be discarded without handling it.
- There are no null values, no implicit conversions, and no reentrancy.

P8. Spec first, then implementation

The reference compiler implements this specification. Where they disagree, one of them has a bug, and the disagreement is adjudicated explicitly and recorded. Every normative rule cites the conformance vectors that exercise it (../04-spec-plan.md describes the suite), and no rule is frozen without vectors.

What vNEXT is not

- **Not backwards compatible.** vNEXT contracts are written for vNEXT. Chapter 4's disposition table maps every Clarity 1–6 construct for porting.
- **Not Turing-complete.** See P1.
- **Not general-purpose.** No I/O, no foreign functions, no off-chain execution.
- **Not privileged.** Clarity is the first language on the chain, not a special one. Everything a Clarity contract does on chain goes through the same chain ABI as any other module.

Footguns removed

This list is the concrete payoff of having no compatibility obligation. Each item is a legacy behavior that vNEXT does not have.

Legacy behavior	vNEXT
`as-contract` grants unrestricted asset access	Only `as-contract?` with explicit allowances

Legacy behavior	vNEXT
`at-block` (already disabled in legacy at epoch 3.4)	Absent
`bit-shift-left` / `bit-shift-right` silently reduce the shift amount modulo 128	Shift amount of 128 or more aborts
Variadic `is-eq`, `and`, `or`, `+`, ... with subtly different typing per arity	Arity is declared: a rest parameter collects extra arguments into a list, typed by the ordinary rules; `and` / `or` are core short-circuit forms; `is-eq` and comparisons are binary only
Hashing an `int` / `uint` hashes an unspecified 16-byte encoding	Hash primitives accept buffers only
`buff-to-int-be` / `buff-to-int-le` zero-pad a short buffer, so `0xff` is `255` but the same bytes in a 16-byte buffer are `-1`	Signed conversions sign-extend from the buffer's actual length; `(buff-to-int-be 0xff)` is `-1`
Trait types written `<name>` require `use-trait` aliasing with a distinct lexical form	Trait types are ordinary type expressions `(trait id)`; aliases use `define-type`
`principal-construct?` returns a tuple with an `error_code` field	`error-code`
Two ways to write tuples, two names for several functions	One way
`(with-all-assets-unsafe)` accepted at runtime inside `restrict-assets?` but rejected by the type checker	Rejected by the grammar; valid only in `as-contract?`
`secp256r1-verify` accepts high-S (malleable) signatures	All ECDSA primitives require low-S
Tuple lub admits a smaller tuple for a larger one, creating hidden fields that must be sanitized at boundaries	Tuple subtyping requires identical field sets
Function arguments may shadow constants, and the checker and evaluator resolve them differently	No shadowing of any in-scope name
`get-tenure-info? block-reward` aborts the transaction during the maturity gap	Returns `none`; maturity defined by coinbase height
Any chain-data lookup the host cannot serve makes the transaction "problematic" and unmineable	All chain-data lookups are total and return `none`
Binding a reserved keyword as a local name passes analysis and fails at runtime	No shadowing, checked at analysis time
A trait defined in the current contract cannot be used in the contract's own top-level expressions	Local definitions are fully usable at the top level
Duplicate parameter names are accepted	Static error

Legacy behavior	vNEXT
A read-only function cannot call a trait function even when every implementation is read-only	Traits may mark functions `read-only`; read-only callers may call those
Constants cannot be used where a literal is required (`as-max-len?` bounds, `contract-call?` targets)	Static constants are usable anywhere a literal is
Trait-typed values cannot be extracted from tuples and called directly	Any trait-typed expression is a valid `contract-call?` target
`is-standard` is true for a contract principal on the current network and false for a standard principal on the other network	`is-current-network` for the network test; `is-standard` is true exactly for standard principals

2. Core Language

This chapter defines the vNEXT core: lexical structure, types, top-level forms, expression forms, functions, and dynamic semantics. Primitive operations are enumerated in 03-primitives.md; everything else a contract author uses is defined in the standard library (04-standard-library.md).

The presentation here is semi-formal. Typing rules are given as prose with signatures. The formal judgment-style rules and the full EBNF are follow-up work once the design questions at the end of this chapter are settled.

2.1 Notation

- `name : (A B) -> R` is a signature: `name` takes an `A` and a `B` and produces an `R`.
- `T`, `E`, `K`, `V`, `A`, `U` are type variables. `N`, `M` are size variables (natural numbers bounding sequence lengths).
- `S <: T` means `S` is a subtype of `T` (2.3.2). `lub(S, T)` is their least upper bound when it exists.
- `never` is the empty type (2.3.1).

2.2 Lexical structure

vNEXT source is a sequence of S-expressions.

- **Comments** begin with `;;` and run to end of line.
- **Identifiers** are non-empty sequences of ASCII letters, digits, and the characters `- _ ! ? + < > = / * ^`, not starting with a digit, at most 128 bytes long. Identifiers are case-sensitive.
- **Integer literals**: `uint` literals are `u` followed by decimal digits (`u42`); `int` literals are optional `-` followed by decimal digits (`42`, `-7`). Values outside the 128-bit range are a parse error.
- **Boolean literals**: `true`, `false`.
- **Buffer literals**: `0x` followed by an even number of hex digits (`0x`, `0xdeadbeef`).
- **ASCII string literals**: `"..."` with escapes `\\`, `\"`, `\n`, `\t`, `\r`. The **`string-ascii` character set** is the printable ASCII characters 0x20 through 0x7E plus tab (0x09), line feed (0x0A), and carriage return (0x0D). A literal may contain only these characters, the three control characters only through their escapes, and no value of a `string-ascii` type ever holds any other byte.
- **UTF-8 string literals**: `u"..."` with the same escapes plus `\u{XXXX}` code point escapes.
- **Principal literals**: standard principals `'SP...` / `'ST...` (c32check encoded); contract principals `'SP....name` or the deployer-relative `.name`; trait identifiers `'SP....contract.trait`, `.contract.trait`, or a bare local trait `name` where a trait identifier is expected. A contract name is 1 to 40 characters, starts with an ASCII letter, and continues with letters, digits, `-`, or `_`. The same limit applies wherever a contract name appears, including `principal-construct?`, `principal-destruct?`, and `principal-to-ascii` (3.5). There are no reserved or special-cased contract names (legacy admitted `__transient`).

- **Tuple literal syntax**: `{ name: expr, ... }`. This is the only tuple constructor syntax.
- `none` is the literal of type `(optional never)`.

Parentheses delimit lists; `{`, `}`, `:`, `,` are used only by tuple syntax.

2.3 Types

2.3.1 Type grammar

```
type      ::= int | uint | bool | principal | never
            | (buff size) | (string-ascii size) | (string-utf8 size)
            | (list size type)
            | (optional type)
            | (response type type)
            | { name: type, ... }              ;; tuple; zero or more fields
            | (trait trait-identifier)
            | alias-name                       ;; introduced by define-type
            | type-variable                    ;; see 2.3.4
            | contract-literal-type            ;; internal; see below

size      ::= uint-literal | size-variable | static-constant-name
```

Meaning of each type:

Type	Values
`int`	signed 128-bit integers
`uint`	unsigned 128-bit integers
`bool`	`true`, `false`
`principal`	standard or contract principals
`never`	no values. The type of `return`, `panic`, and the "other side" of `ok` / `err` / `none`; it stays in a type that nothing widens
`(buff N)`	byte sequences of length at most `N`
`(string-ascii N)`	strings of at most `N` bytes from the `string-ascii` character set (2.2)
`(string-utf8 N)`	UTF-8 strings of at most `N` code points
`(list N T)`	sequences of at most `N` values of type `T`
`(optional T)`	`none` or `(some v)` with `v : T`
`(response T E)`	`(ok v)` with `v : T` or `(err e)` with `e : E`
`{ f: T, ... }`	records with exactly the listed fields

Type	Values
`(trait X)`	contract principals whose interface implements trait `X`
contract literal	the one contract principal named by a literal `'SP....c` or `.c`, or by a static constant holding one (2.4.1). Internal: it cannot be written in source, is never an interface or storable type, and exists so that a literal can be used wherever a trait type is expected (2.3.2) and as a `contract-call?` target (2.5.8)

`(buff N)`, `(string-ascii N)`, `(string-utf8 N)`, and `(list N T)` are the four **sequence types**. Their **element types** are `(buff 1)`, `(string-ascii 1)`, `(string-utf8 1)`, and `T` respectively.

Every type has a statically known maximum serialized size (defined in the chain ABI's wire format). Two limits (2.8) apply to it, depending on where the type occurs:

- The **value size limit** applies to the type of every name (a `let` or `match` binding, or a parameter of a function that is not inline), every function result, every type written in source, and every value written to storage, emitted in an event, or passed to `contract-call?`.
- The **intermediate size limit** applies to the type of every other expression: a primitive's argument, a list or tuple under construction, or an inline function's argument before it is bound. Inline arguments are checked after expansion, at the position where they land.

A type that exceeds the limit for its position is a type error. So no value larger than the value size limit can be bound, returned, or stored, and a value that lives only during the evaluation of one expression is bounded by the intermediate limit. `replace-at?`'s `(concat (concat prefix (list item)) suffix)` has bound `2N + 1` before `as-max-len?` brings it back to `N`, which fits the intermediate limit even when `N` is near the value limit. Type nesting depth MUST NOT exceed the type depth limit.

2.3.2 Subtyping

Subtyping formalizes what legacy Clarity calls "admission". It is the smallest reflexive, transitive relation such that:

- `never <: T` for every `T`.
- `(buff N) <: (buff M)`, `(string-ascii N) <: (string-ascii M)`, `(string-utf8 N) <: (string-utf8 M)` when `N <= M`.
- `(list N S) <: (list M T)` when `N <= M` and `S <: T`.
- `(optional S) <: (optional T)` when `S <: T`.
- `(response S1 S2) <: (response T1 T2)` when `S1 <: T1` and `S2 <: T2`.
- `{ f1: S1, ... } <: { f1: T1, ... }` when both have exactly the same field names and `Si <: Ti` for every field.
- The contract literal type of `c` (2.3.1) `<: (trait X)` when the interface of `c` implements `X` (chain ABI trait rules), and `<: principal` always.
- `(trait X) <: (trait Y)` when every function required by `Y` is present in `X` with an identical signature (structural sub-trait).
- `(trait X) <: principal`.

`lub(S, T)` is defined pointwise by the same rules (taking the larger bound, the lub of components, and so on) and is undefined when `S` and `T` have different shapes (for example `int` and `uint`, or tuples with different field sets). An undefined lub where one is required is a type error.

Among the principal-like types the join is always defined, and it is the most specific type the rules above name, falling back to `principal`:

- `lub(S, S) = S`.
- `lub(S, (trait X)) = (trait X)` when `S <: (trait X)`; symmetrically for the other argument.
- Otherwise `lub(S, T) = principal`, including two different contract literals, two trait types neither of which is a sub-trait of the other, and anything with `principal`.

The join of two unrelated trait types is `principal` rather than an anonymous trait of their common functions, because such a trait has no name, could not be written in an interface, and would give a value a type no one declared. A list of contract literals meant as trait values therefore states the trait once: `(list (as (trait X) .a) .b .c)` has type `(list 3 (trait X))` when all three implement `X`.

Because tuple subtyping requires identical field sets, every runtime tuple has exactly the fields of its static type. There are no hidden fields, so no value ever needs to be re-shaped ("sanitized") when it crosses a function or contract boundary.

2.3.3 Storable types and interface types

A type is **storable** if it contains no trait type and no type or size variable. Only storable types may appear in `define-data-var`, `define-map`, `define-non-fungible-token`, and in `from-consensus-buff?`. A storable type may contain `never`; such a position can hold no value, which is harmless.

Excluding trait types keeps every call target traceable to the transaction: a trait-typed value can only come from a contract literal or a public function's argument, so the contracts a transaction can reach are named in its payload or in the code, never in chain state. A registry therefore stores principals and has the caller name the target:

```
(define-map extensions uint principal)

(define-public (run (id uint) (ext (trait extension)))
  (begin
    (asserts! (is-eq (map-get? extensions id) (some ext)) (err u1))
    (contract-call? ext execute)))
```

`(trait extension) <: principal` (2.3.2), so the comparison needs no conversion. The restriction is not needed for termination or cost bounds (a call through a trait parameter already has a statically unknown callee). It is kept because lifting it later is backward compatible and restoring it would not be.

A type is an **interface type** if it contains no type or size variable. Trait types are allowed. Only interface types may appear in the parameter and result types of public and read-only functions and in trait definitions.

An interface type may contain `never`. A public function whose body is `(ok true)` has the result type `(response bool never)`, which states that the function cannot return `err`. The chain ABI's type grammar includes `never`, and boundary validation (2.5.8) admits no value for it. An author who wants a wider type in the interface states it with `as` (2.5.10): `(as (response bool uint) (ok true))`.

2.3.4 Type variables and size variables

An identifier in type position that is not a built-in type name, a `define-type` alias, or a trait identifier denotes a **type variable**. An identifier in size position that is not a literal denotes a **size variable**. Both are implicitly universally quantified over the definition in which they appear, and each MUST occur in at least one parameter type of that definition.

Type and size variables may appear only in `define-private` and `define-inline` definitions. Public and read-only functions, traits, storage definitions, and constants are monomorphic.

> [!NOTE] Size arithmetic (`N + M`, `N + 1`, `K × M`) appears only in primitive signatures in 03-primitives.md. Contract authors never write it, because function result types are inferred, not declared.

A **static constant** (2.4.1) of type `uint` may be used as a size, and a static constant of contract principal type may be used wherever a contract principal literal is required.

2.3.5 Type aliases

`(define-type name type)` introduces a transparent alias. The alias is interchangeable with its definition everywhere. Aliases may not be recursive (2.4.3) and may not contain type or size variables.

This form replaces legacy `use-trait`: `(define-type sip010 (trait 'SP....sip-010-trait.sip-010-trait))`.

2.3.6 Static reference kinds

Some parameters do not hold runtime values but refer statically to a definition in the current contract. These **reference kinds** are not types: they cannot appear in `let` bindings, storage, interfaces, tuples, or sequences.

```
ref-kind  ::= (var type)
            | (map type type)
            | (fungible-token)
            | (non-fungible-token type)
            | (function (type ...) type)
```

An argument in a reference-kind position MUST be either the name of a definition of the matching kind in the current contract, or a parameter of the same kind in the enclosing function. References are resolved at instantiation time (2.6.2) and have no runtime representation.

Reference kinds let one helper serve every map (or variable, or token) of a given shape; each distinct reference is its own instance, so which storage a function touches stays statically known. Reference kinds may appear only in `define-private` and `define-inline` parameters. `(function ...)` arguments may name any function defined in the current contract or the prelude except inline functions, or any primitive that is not a special form.

2.4 Contracts and top-level forms

A contract is a sequence of top-level forms. A definition may refer to any other definition in the contract, earlier or later, as long as the definition graph is acyclic (2.4.3). Source order matters only for code evaluated at deploy (2.4.2).

2.4.1 Definition forms

Form	Introduces	Rules
`(define-constant name expr)`	A named value	`expr` is deploy-time code (2.4.2), evaluated once; its type is inferred and MUST satisfy the value size limit (2.3.1). The type may contain `never`, so `(define-constant ERR-NOT-FOUND (err u404))` has type `(response never uint)` and is usable in any function whose error type is `uint`. If `expr` is a literal, or is built only from literals and other static constants using primitives that read neither state nor context (arithmetic, sequence, conversion, and hash primitives), the constant is **static**: its value is known at analysis time and it may be used wherever a literal is required (size terms, `contract-call?` targets, `as-max-len?` bounds)
`(define-type name type)`	A type alias	2.3.5
`(define-data-var name type expr)`	A persistent variable	`type` storable; `expr : T <: type` is deploy-time code (2.4.2)
`(define-map name key-type value-type)`	A persistent map	both types storable
`(define-fungible-token name)` and `(define-fungible-token name expr)`	A fungible token, optionally with a total supply cap `expr : uint`	
`(define-non-fungible-token name type)`	A non-fungible token with identifier type `type`	`type` storable

Form	Introduces	Rules
`(define-trait name ((fn-name (type ...) type) ...))` and `(define-trait name ((fn-name (type ...) type read-only) ...))`	A trait: a set of function names with interface-typed signatures whose result types are `response` types. A function marked `read-only` may only be implemented by a read-only function, and may be called through the trait from read-only code (2.7.4)	
`(impl-trait trait-identifier)`	A conformance claim, checked statically: every function in the trait MUST be defined as a public or read-only function with a compatible signature (parameters contravariant, result covariant), and a function the trait marks `read-only` MUST be defined with `define-read-only`	
`(define-public (name (param type) ...) expr ...)`	An externally callable function	parameter types are interface types; result type MUST be a `response`; monomorphic
`(define-read-only (name (param type) ...) expr ...)`	An externally callable function that MUST NOT mutate state (2.7.4)	parameter types are interface types; monomorphic
`(define-private (name (param type-or-ref-kind) ...) expr ...)`	An internal function called by value	may be generic and overloaded, and may end with a rest parameter (2.6)
`(define-inline (name (param type-or-ref-kind) ...) expr ...)`	An internal function expanded at each call site, with call-by-name parameters (2.6.3)	may be generic and overloaded

Parameter names within one definition MUST be distinct (a duplicate is a static error, not a runtime one). A function body is a non-empty sequence of expressions evaluated in order; the value of the last is the value of the body. Result types are inferred (2.6.4).

Intermediate responses MUST be handled. In any expression sequence (a function body, a `let` or `begin` body, or a `match` arm), every expression other than the last MUST NOT have a `response` type. A `response` is consumed by returning it, matching on it, or passing it to a function; it cannot be dropped (P7). This is a static error.

2.4.2 Top-level expressions

Deploy-time code is every `define-constant` initializer, `define-data-var` initializer, `define-fungible-token` supply cap, and top-level form that is not a definition. It is evaluated once, at deploy, in source order. Deploy-time code is ordinary code: it may read and write state, call other contracts, and emit events, subject only to the `return` restriction below.

Definitions used at deploy must come first. Deploy-time code MUST NOT use, directly or through the definitions it calls (2.4.3), a definition that has an initializer and appears later in the source: a constant that is not static (2.4.1), a data variable, or a fungible token with a supply cap. This is a static error. Every other kind of definition (functions, types, traits, maps, tokens without a cap, and static constants) may be used from any position, because nothing about it is evaluated at deploy: a static constant's value is substituted at analysis time, so its position in the file is irrelevant.

The contract's own definitions, including its traits, are fully usable in top-level expressions: a trait defined in the contract is resolved against the contract's own analysis, not against a deployed copy of the contract, so `(f .other-contract)` with `f` taking a locally defined trait type is valid at the top level. `return` MUST NOT appear in a top-level expression, so the early-exit library functions (`unwrap!`, `try!`, `asserts!`, `unwrap-err!`), which expand to `return`, are unusable in top-level expressions and in `define-constant` and `define-data-var` initializers; `unwrap-panic` and `match` serve there. A runtime error in any top-level expression aborts the deploy.

2.4.3 The definition graph is acyclic

Definition `a` depends on `b` if the body, type, or initializer of `a` mentions the name `b` anywhere: in call position, in type position, or as a reference argument. The dependency graph over all definition names in a contract MUST be acyclic. This single rule excludes recursion (direct, mutual, and through function references), recursive types, and recursive constants.

The graph is over names, not instances, and is computed from the source alone, before overload resolution and instantiation (2.6.2). All definitions of an overloaded name are one node, so one overload of a name MUST NOT use another; the shared part lives in a helper, as the prelude's `std-sqrti-uint` does for the two `sqrti` overloads. Because the graph is acyclic before instantiation begins, every chain of instantiations and inline expansions is finite (P1), and the deploy-order check of 2.4.2 can be made on the source.

2.4.4 Contract interface

The compiler emits the contract's interface into its module in the chain ABI's interface format: its public and read-only functions with their signatures and read-only marks, its implemented traits, its defined traits, and its token definitions. The chain validates the interface against the module at deploy. Other contracts, whatever their source language, compile calls against this interface only, and the chain validates every call against it at runtime (2.5.8).

2.4.5 Standard library names

The standard library prelude (std/prelude.clar) is in scope in every contract as if it were part of the contract's own top-level forms. A contract MUST NOT define a top-level name that the prelude defines.

2.5 Expression forms

Beyond literals, names, and function application, the core has the following special forms. Anything not listed here is either a primitive (03-primitives.md) or a library definition.

2.5.1 Names

A bare identifier denotes a `let`- or `match`-bound variable, a function parameter, or a `define-constant`; because there is no shadowing (2.5.2), at most one of these applies at any point. Function, map, variable, and token names are not values and may appear only in application or reference-argument position. There are no other bare names: the execution context is read with `get-current-context` (3.9).

2.5.2 `let`

```
(let ((name expr) ...) body-expr ...)
```

Bindings are evaluated left to right; each binding is in scope for the following bindings and the body. The type is the type of the last body expression.

No shadowing. A `let` binding, a `match` binding, or a function parameter MUST NOT reuse a name that is already in scope, including the names of top-level definitions, prelude definitions, and primitives. Every name therefore has exactly one meaning at each point in the program, and the type checker and the evaluator cannot disagree about which one applies.

Legacy is equally strict about locals, top-level definitions, and builtin names. Legacy source that uses a name reserved only in vNEXT (such as `max`, `min`, `abs`, `fold-left`, or `as`) as a local must rename it when ported. The reserved names are fixed per language version, because the prelude is part of the language version (chapter 4).

`(begin expr ...)` is sugar for `(let () expr ...)`.

2.5.3 `match`, `and`, `or`

```
(match expr arm arm)
arm     ::= (pattern body-expr ...)
pattern ::= (some name) | none | (ok name) | (err name) | true | false
```

`match` is the core's branching construct. The scrutinee type determines the required patterns, each of which MUST appear exactly once:

Scrutinee type	Required patterns	Bindings
`(optional T)`	`(some x)`, `none`	`x : T`
`(response T E)`	`(ok x)`, `(err e)`	`x : T`, `e : E`
`bool`	`true`, `false`	none

The type of the `match` is the lub of the arm types. Exactly one arm is evaluated.

`if` is a library function defined over `match` on `bool`.

```
(and expr expr ...)
(or expr expr ...)
```

`and` and `or` take two or more `bool` expressions and evaluate them left to right, stopping at the first `false` (`and`) or the first `true` (`or`). They are core forms, not library functions, because short-circuiting over any number of arguments cannot be written in the language: an inline function has a fixed number of by-name parameters, and a rest parameter (2.6.5) is a list value whose elements are all evaluated before the body runs. `(and a b c)` means exactly `(match a (true (match b (true c) (false false))) (false false))`.

2.5.4 `return`

```
(return expr)
```

Evaluates `expr`, then terminates the innermost enclosing function body (after inline expansion, 2.6.3) with that value. `return` has type `never`. The type of `expr` contributes to the enclosing function's inferred result type (2.6.4). `return` MUST NOT appear in a top-level expression.

When a `return` crosses an `as-contract?` or `restrict-assets?` boundary, the rules in 2.5.8 apply.

`unwrap!`, `try!`, `asserts!`, and the rest of the early-exit family are library functions defined over `match` and `return`.

2.5.5 Application

```
(f arg ...)
```

`f` is a primitive, a function defined in the current contract or the prelude, or a `(function ...)` parameter. Arguments are evaluated left to right (except for inline functions, 2.6.3). Typing and overload resolution are defined in 2.6.2.

2.5.6 Constructors

`(some e)`, `(ok e)`, `(err e)`, `(list e ...)`, and `{ f: e, ... }` construct values.

Form	Type
`(some e)` with `e : T`	`(optional T)`
`(ok e)` with `e : T`	`(response T never)`
`(err e)` with `e : E`	`(response never E)`
`(list e1 ... en)`	`(list n lub(T1, ..., Tn))`; `(list)` is `(list 0 never)`
`{ f1: e1, ... }`	`{ f1: T1, ... }`; field names MUST be distinct

`list`, `and`, and `or` are the only variadic forms. Functions take a variable number of arguments only through a rest parameter (2.6.5).

2.5.7 Tuple access

- `(get f e)` with `e : { f: T, ... }` has type `T`. `f` MUST be a field of the tuple type.
- `(merge e1 e2)` with tuple types `R1`, `R2` has the tuple type whose fields are the union of both, taking the type from `R2` for fields present in both. At runtime, fields of `e2` overwrite fields of `e1`.

2.5.8 Cross-contract calls

```
(contract-call? contract function-name arg ...)
```

`contract` is a contract principal literal, a static constant (2.4.1) of contract principal type, or an expression of a trait type `(trait X)`; in the trait case `function-name` MUST be a function of `X`. Any expression of trait type is acceptable, including a tuple field or a `let`-bound name. Arguments are type-checked against the callee's stored interface (literal case) or the trait signature (trait case). In the literal case the callee's interface is therefore an input to compilation, and the compiler records it in the module's provenance (../03-architecture.md section 2). The result type is the callee's declared result type. The callee MUST be a public or read-only function and MUST NOT be in the current contract.

The call is dispatched by the chain, which enforces the depth limit and the no-reentrancy rule (2.7.5). The callee may be written in any source language. The chain validates the arguments against the callee's declared parameter types and the result against its declared result type, and for a trait target checks that the principal's interface implements the trait. A Clarity contract can therefore rely on its parameter types whoever calls it.

2.5.9 Context scoping and asset allowances

```
(as-contract? (allowance ...) body-expr ...)
(restrict-assets? owner-expr (allowance ...) body-expr ...)

allowance ::= (with-stx amount)
            | (with-ft contract token-name amount)
            | (with-nft contract asset-name identifier-list)
            | (with-staking amount)
            | (with-pox)
            | (with-all-assets-unsafe)   ;; as-contract? only
```

Both forms evaluate their body in a nested state frame and, on completion, compare the asset movements of a designated owner against the allowance list.

- `as-contract?` sets the context's `sender` and `caller` fields (3.9) to the current contract for the body; the designated owner is the current contract.
- `restrict-assets?` leaves the context unchanged; the designated owner is the value of `owner-expr : principal`.

Both have type `(response T uint)` where `T` is the body type. Outcome:

1. If the body completes normally and every movement of the owner's assets is covered by an allowance, the frame is committed and the value is `(ok body-value)`.
2. If any movement is not covered, the frame is rolled back and the value is `(err i)`, where `i` is the index of the first violated allowance (defined precisely in the semantics chapter).
3. If the body is exited by a `return` that crosses the form, the allowance check still runs. On violation, the frame is rolled back, the `return` is cancelled, and the form's value is `(err i)`. Otherwise the frame is committed and the `return` proceeds.
4. A runtime error in the body aborts the transaction as usual.

Allowance sub-forms are valid only in allowance position. `(with-all-assets-unsafe)` is valid only in `as-contract?`; in `restrict-assets?` it is a type error, since it would make the form a no-op. The other allowances' arguments are ordinary expressions evaluated before the body. The number of allowances is bounded (2.8).

2.5.10 Type ascription

```
(as type expr)
```

`type` is a type expression (2.3.1), and may use the type and size variables in scope. `expr`'s type MUST be a subtype of `type` (2.3.2); the form has type `type` and evaluates to the value of `expr`. `as` only widens: it never narrows, converts, or checks anything at runtime. It exists to give an initial value the type a fold needs (2.6.2), and to give a type error a place to point: `(as (list 10 uint) (list))`, `(as (optional principal) none)`, `(as (string-ascii N) "")`.

2.6 Functions

2.6.1 Visibility

Kind	Callable from other contracts	Callable internally	May mutate state	May be generic or overloaded	Evaluation
`define-public`	yes	yes	yes	no	by value
`define-read-only`	yes	yes	no	no	by value
`define-private`	no	yes	yes	yes	by value
`define-inline`	no	yes	yes	yes	expanded at call site, by name

A public function called from another contract, or as the entry point of a transaction, has the rollback semantics of 2.7.3. Public functions called internally behave like private functions.

2.6.2 Instantiation and overload resolution

Let a call be `(f a1 ... an)`, and let the candidate definitions of `f` be those that accept `n` arguments: exactly `n` parameters, or, for a rest-parameter definition, fewer than `n` fixed parameters (2.6.5). A call whose last argument is a spread `e...` is accepted only by rest-parameter definitions with exactly `n − 1` fixed parameters. For a candidate with parameter types `P1 ... Pn` (which may contain type variables, size variables, and reference kinds) and free variables `X`, an **instantiation** is a substitution `σ` over `X` such that:

- for each value parameter, `type(ai) <: Pi[σ]`;
- for each reference-kind parameter, `ai` is a name (or reference parameter) of the matching kind whose declared types match `Pi[σ]`; for `(function (Q ...) R)`, the named function MUST itself admit an instantiation whose parameter types are supertypes of `Q[σ]...` and whose result type is a subtype of `R[σ]`. A rest-parameter definition with `k` fixed parameters is named at a function type with `k + 1` parameters, and the last one fills its rest parameter as a one-element list (2.6.5).

The candidate **admits** the call if an instantiation exists. Among the instantiations that exist, the **least** one (pointwise under `<:`, and smallest size values) is chosen; if no least instantiation exists the candidate does not admit the call. This is the declarative form of the familiar "unify and take the least upper bound" algorithm.

When a type variable occurs in both a parameter and the result of a `(function ...)` parameter, as the accumulator `A` does in `fold-left`, the least instantiation is a least fixpoint rather than a unification: the initial value's type is a lower bound for `A`, and the named function's result type at that `A` is another, so `A` is the least type closed under both. The prelude relies on this: `(fold-left std-prepend-byte 0x`

`bytes)` starts from `(buff 0)` and settles at `(buff 16)`, and `index-of?` starts from `none` and settles at `(optional uint)`. Every type in the language has finitely many supertypes below the size limit, so the iteration terminates, but a step function that grows its accumulator would take as many steps as the size limit allows before failing. The iteration is therefore capped: starting from `A0`, the initial value's type, each step computes `A(i+1) = lub(A(i), R(A(i)))`, where `R` is the named function's result type, and if `A(i+1) = A(i)` is not reached within the fixpoint step limit (2.8), the candidate does not admit the call. Every prelude fold closes in two steps. A fold that needs a wider starting type states it with `as` (2.5.10): `(fold-left f (as (list 10 uint) (list)) xs)`.

Overload resolution: exactly one candidate MUST admit the call. Zero is a type error; more than one is an ambiguity error. Overloading is by arity and by parameter types; there is no ranking. Because `never <: T` for every `T`, an argument of type `never` is admitted by every candidate, so `(unwrap! (panic) e)` is an ambiguity error; the call could not complete anyway.

The result type of the call is the candidate's inferred result type under `σ`, with size arithmetic in primitive signatures evaluated on the concrete sizes. A resulting size that exceeds the limit for the call's position (2.3.1) is a type error.

Each distinct `(candidate, σ, reference arguments)` triple is a distinct **instance**. Instances are what the compiler operates on; the definition graph (2.4.3) is over names and is checked before any instance exists. Instance bodies are type-checked with the concrete substitution applied ("template" semantics): a generic body that is never instantiated is parsed and scope-checked but not type-checked.

A type error found while checking an instance body is reported at the call site that created the instance. The report MUST name the called definition and the instantiation (for example, `max` at `bool`), and MAY include the location inside the body as secondary information. When the instance was created from inside another instance body, the report MUST include the chain of call sites back to the user's code.

Template semantics give functions over "any tuple with these fields" without a separate construct:

```
(define-private (credit (acct T) (amt uint))
  (merge acct { balance: (+ (get balance acct) amt) }))
```

`credit` admits any tuple with a `uint` field `balance` and returns a tuple of the same shape. Each tuple shape is its own instance, so no value carries fields its static type lacks (2.3.2). A call with a tuple that has no `balance` field is reported as `credit` at that tuple type, at the call site.

2.6.3 Inline functions

An inline function is expanded at each call site:

1. The body is type-checked at the call site with parameters bound to the argument expressions' types (2.6.2).
2. Evaluation of the call is evaluation of the body with each parameter reference replaced by its argument expression, evaluated in the caller's context at the point of reference (call by name). An argument whose parameter is never referenced on the taken path is not evaluated. Because

evaluation happens at the point of reference, an argument referenced inside an `as-contract?` or `restrict-assets?` form in the body is evaluated inside that frame, with the frame's `sender` and `caller`, and the order in which arguments are evaluated is the order of their references in the body, not their order at the call site. Prelude inline functions reference their parameters outside any such frame. Prelude functions whose names read as ordinary calls (`default-to`, `map-insert?`) are private, so their arguments are evaluated eagerly as in legacy; laziness is confined to control-flow forms, and the library documents each place where it changes legacy behavior (`unwrap!`, `unwrap-err!`).

3. Names bound inside the body (`let`, `match`) are fresh and cannot capture names in argument expressions. Free names in the body resolve in the defining contract's top-level scope.
4. `return` inside the body returns from the caller's innermost enclosing function body, not from the inline function.

To make call by name safe, each **value** parameter of an inline function MUST be referenced at most once in the body (syntactically). A body that needs a parameter twice binds it with `let` first. Reference-kind parameters have no evaluation and may be referenced any number of times.

Inline functions cannot be passed as `(function ...)` arguments. Inline expansion is finite because the definition graph is acyclic (2.4.3).

> [!NOTE] **When to use `define-inline`.** An inline function is the only way to write a helper that returns from its caller or evaluates an argument only on some paths. Use it for exactly those two cases, and use `define-private` for everything else. For example, a helper that replaces an error code and exits in one step cannot be private:
>
> ``` (define-inline (try-with! (value (response T E)) (code F)) (match value ((ok inner) inner) ((err ignored) (return (err code)))))
>
> (define-public (withdraw (amount uint)) (let ((balance (try-with! (get-balance (get sender (get-current-context))) ERR-NO-ACCOUNT))) ...)) ```
>
> A private function is the better default because it behaves like every other call: each argument is evaluated exactly once, left to right, before the body runs. It can be passed to `map`, `filter`, and the folds. It is compiled once per instance instead of once per call site. Its body may use a parameter any number of times.
>
> An inline function gives those up. At a call site, an argument whose parameter is not used on the path taken is never evaluated, so its side effects (a `contract-call?`, a `var-set`, an abort) do not happen. Arguments are evaluated in the order the body uses them, not the order they are written. An argument used inside an `as-contract?` or `restrict-assets?` form in the body sees that frame's `sender`. A function whose name reads as an ordinary call should not be inline unless its documentation states which arguments may go unevaluated.

2.6.4 Result type inference

The result type of a function instance is the lub of the type of its last body expression and the types of the operands of every `return` reachable in its body (after inline expansion). If the lub is undefined, it is a type error. For `define-public`, the result type MUST be a `response` type.

The result type is not declared, and may contain `never` (2.3.3). A function with no `err` path has an error type of `never`; a function with no `ok` path has a value type of `never`. Either is a valid interface type and is what the compiler emits in the module's interface (2.4.4).

2.6.5 Rest parameters

The last parameter of a `define-private` definition MAY be a **rest parameter**, written with `...` after its type:

```
(define-private (name (param type) ... (rest T...)) expr ...)
```

A call supplies the fixed parameters in order and **one or more** further arguments for the rest parameter. Insides the body, the rest parameter is a list: `rest : (list N T)`, where `N` is the number of rest arguments at that call site. The call `(name a1 ... ak b1 ... bm)` is the call of the definition with `a1 ... ak` for the fixed parameters and `(list b1 ... bm)` for the rest, so arguments are evaluated left to right and the list is typed by 2.5.6. Each call site is its own instance (2.6.2), with `N` bound to `m`.

A function that takes "two or more" is one definition. Its binary case is simply the call with one rest argument:

```
(define-private (std-max-pair (left T) (right T))
  (if (> left right) left right))

(define-private (max (left T) (right T...))
  (fold-left std-max-pair left right))

;; (max a b)     is (fold-left std-max-pair a (list b))
;; (max a b c d) is (fold-left std-max-pair a (list b c d))
;; (max a)       is an arity error: a rest parameter needs an argument
```

`fold-left` passes the accumulator first (3.3), so this pattern is the left fold of the helper whether or not the helper is commutative.

An argument that is already a list is not spread implicitly: in `(max a xs)` with `xs : (list 5 int)`, `xs` is a single rest argument, so `right : (list 1 (list 5 int))` and the body fails to type-check. To pass an existing list as the rest parameter, **spread** it by writing `...` after the argument:

```
(max a xs...)                            ;; right is xs
(concat "0x" (map std-hex-byte bytes)...)  ;; one string from many
```

A spread argument `e...` with `e : (list N T)` MUST be the only argument in the rest position: the call supplies the fixed parameters and then `e...`, with no other rest arguments. The rest parameter is the

value of `e`, typed `(list N T)`, and the instance binds `N` to that bound. `e` is evaluated once, in its position, like any other argument. Unlike a call without a spread, the rest list's runtime length may be less than `N`, including zero, so `(max a (list)...)` is `a`. Only a rest parameter admits a spread argument: a definition with a fixed parameter in that position does not accept the call. Spread is not available for special forms (`and`, `or`, `let`, `match`).

Rules:

- **All fixed or all rest.** If any definition of a name has a rest parameter, every definition of that name MUST have one, with the same number of fixed parameters. Overloads then differ only by type, and overload resolution needs no rule about arity.
- **Passing a rest function.** A rest-parameter function may be passed as a `(function ...)` argument at the type with one more parameter than its fixed parameters; the last argument becomes a one-element rest list (2.6.2). So `(fold-left max 0 xs)` and `(fold-left + 0 xs)` work.
- **No self-reference.** A rest-parameter body cannot fold itself, since that is a cycle (2.4.3). The binary operation lives in a helper, as `std-max-pair` above, or in a primitive.
- **Private only.** Public and read-only functions MUST NOT have rest parameters, because their interface has a fixed arity in the chain ABI. Inline functions MUST NOT have them either: rest arguments are evaluated as a list before the body runs, which would silently make an inline function strict in those arguments.
- **Primitives** whose chapter 3 signature ends in `I...` or `S...` follow the same rules, including spread. Their result type may be tighter than a user definition could state; `concat`'s bound, for example, is the exact sum of its arguments' bounds, or `N + K × M` when a `(list K S[M])` is spread after a first argument of bound `N`.

A compiler instantiates each call site with its own `N`, so it can compile the one-argument case of a rest function without building a list. The spec does not require it.

`is-eq` and the comparisons have no rest parameter: `(is-eq a b c)` has no single obvious meaning, and "all equal" is `(and (is-eq a b) (is-eq a c))`.

2.7 Dynamic semantics

2.7.1 Evaluation

Evaluation is strict and left to right, except that inline parameters are evaluated by name (2.6.3) and `match` evaluates exactly one arm. Every expression evaluates to a value of its static type or aborts. A runtime value carries no static type information of its own; its size and shape are determined entirely by the value, and a value passed to a callee is exactly the value the caller computed.

2.7.2 Runtime errors

A runtime error (arithmetic overflow, division by zero, failed conversion, `panic`, cost or memory limit exceeded, and the other conditions listed per primitive) aborts the entire transaction. There is no mechanism to catch a runtime error. All state changes of the transaction are discarded; the transaction still pays for the cost it consumed.

2.7.3 Public function results and rollback

When a public function is invoked from another contract or as a transaction entry point, its state changes (variables, maps, token balances, events) are buffered. If it returns `(ok v)`, they are committed to the caller's frame. If it returns `(err e)`, they are discarded and `(err e)` is returned to the caller, which continues executing. A `return` of an `err` value has the same effect as evaluating to it. The chain performs the commit or discard based on the returned value, so the rule holds for calls between modules of any source language.

2.7.4 Read-only enforcement

A read-only function MUST NOT, directly or through functions in the same contract, execute `var-set`, `map-set`, `map-delete`, any `*-mint?`, `*-burn?`, or `*-transfer?` primitive. A `contract-call?` from read-only code is permitted only when the callee is statically known to be read-only: a literal or static constant target whose interface marks the function read-only, or a trait target whose trait marks the function `read-only`. All of this is checked at compile time, so a violation is reported before deploy. The chain independently rejects any write attempted inside a read-only call frame, which is what makes the guarantee hold when read-only code calls a module from another language.

2.7.5 Depth and reentrancy

The chain rejects a `contract-call?` into a contract that already has an active call frame in the current transaction, and rejects any call that would exceed the call depth limit. Both are runtime errors.

2.7.6 Events

`print` and every asset primitive emit events into the transaction's event batch. Events of a rolled-back frame are discarded with it.

2.7.7 Cost

The language defines no costs (P4). A contract runs as the Wasm module its source compiles to, and its execution cost is the fuel that module consumes plus the chain's charges for the host functions it calls. Library functions are compiled and metered like any other code. Exceeding the transaction or block budget is a runtime error.

Fuel replaces a per-construct runtime cost table. It does not replace the chain's storage dimensions: reads and writes are still counted and sized, with the budget dimensions defined in the chain ABI.

The following rules are about resource safety, not pricing, and bind every compiler:

- Memory for a decoded value is reserved against the size bound of its result type before decoding begins. `from-consensus-buff?` reserves `serialized-size(T)`, so a hostile input cannot allocate more than its declared type admits.
- A trait-typed value is a principal and is sized as one.
- Compilation happens off chain, so its cost is the deployer's. What the chain charges for is the module: its size and its validation at deploy, as defined by the chain ABI.

2.8 Limits

These are language-level limits. Hosts MAY impose stricter limits on contract source size and transaction budgets.

Limit	Value	Inherited from
Maximum serialized value size	1,048,576 bytes	`MAX_VALUE_SIZE`
Maximum intermediate value size (2.3.1)	8,388,608 bytes	new, provisional; MUST stay below the chain's Wasm memory limit
Fixpoint step limit for fold accumulators (2.6.2)	8	new, provisional
Maximum type nesting depth	32	`MAX_TYPE_DEPTH`
Maximum identifier length	128 bytes	`MAX_STRING_LEN`; string literals are bounded only by the value size limit, as every other value is
Maximum call depth (functions and contract calls)	128	`MAX_CALL_STACK_DEPTH`
Maximum allowances per `as-contract?` / `restrict-assets?`	128	`MAX_ALLOWANCES`

There is no limit on the size of a contract after inline expansion and instantiation, which can grow exponentially (definitions that each call the next twice, or instantiate it at two types). The only limit is the chain's limit on module size. A compiler MUST report a contract whose module would exceed it as a compile error, and SHOULD stop expanding as soon as the limit is passed. A language-level cap would not make validity compiler-independent, since near the limit whether a contract fits already depends on code generation (P5).

3. Primitive Operations

This chapter enumerates every primitive in the vNEXT core. A primitive is an operation that passes the layering rule in README.md: it needs the host, it cannot be expressed in the language, it is a cryptographic standard, or it is a base operation on a type's representation. Everything a contract author uses that is not listed here or in the special forms of 02-core-language.md is a standard-library definition.

Every primitive has a declared arity (P6). A signature whose last parameter is written `I...` or `S...` has a rest parameter (2.6.5): the primitive takes its fixed arguments plus one or more further arguments, or a single spread list (2.6.5), and the result is the left fold of the operation over them, so `(- a b c)` is `(- (- a b) c)`, `(- a b)` is ordinary subtraction, and `(+ 0 xs...)` is the sum of `xs`. Passed as a function argument, such a primitive is its two-argument operation: `(fold-left + 0 xs)`.

Notation otherwise follows 2.1: uppercase letters are type or size variables; "for `I` in {...}" denotes a family with one signature per listed type. Subtyping (2.3.2) applies to arguments, so a `(buff 4)` is accepted where `(buff 16)` is written.

Each entry gives the signature, semantics, and error conditions. Primitives have no cost entries (P4). A primitive the compiler lowers to Wasm costs the fuel of that code; a primitive that calls a host function is charged by the chain ABI. Which primitives are host functions is decided by the chain ABI and the compilation chapter, not here.

3.1 Arithmetic

For `I` in { `int`, `uint` }:

Primitive	Signature	Semantics	Runtime errors
`+`	`(I I...) -> I`	addition	overflow
`-`	`(I I...) -> I`	subtraction	overflow/underflow
`*`	`(I I...) -> I`	multiplication	overflow
`/`	`(I I...) -> I`	division truncating toward zero	divisor is zero; overflow (the most negative `int` divided by `-1`)
`to-int`	`(uint) -> int`	conversion	argument at least 2^127
`to-uint`	`(int) -> uint`	conversion	argument negative

Primitive	Signature	Semantics	Runtime errors
`bit-and`	`(I I...) -> I`	bitwise and	
`bit-or`	`(I I...) -> I`	bitwise or	
`bit-xor`	`(I I...) -> I`	bitwise exclusive or	
`bit-shift-left`	`(I uint) -> I`	shift left, discarding bits shifted out; the result reinterprets the 128-bit pattern as `I`	shift amount 128 or more
`bit-shift-right`	`(I uint) -> I`	shift right; arithmetic for `int`, logical for `uint`	shift amount 128 or more

Library, not primitive: `mod`, `pow`, `sqrti`, `log2`, `bit-not`, `max`, `min`, `abs`. Their error conditions are the same as legacy and are defined by their source.

3.2 Comparison and equality

For `C` in {`int`, `uint`, `(buff N)`, `(string-ascii N)`, `(string-utf8 N)`}:

Primitive	Signature	Semantics
`<`, `<=`, `>`, `>=`	`(C C) -> bool`	numeric order for integers; lexicographic byte order for sequences

Primitive	Signature	Semantics
`is-eq`	`(T T) -> bool`	structural equality. Both arguments are admitted at `lub` of their types; an undefined lub is a type error

3.3 Sequences

`S` ranges over the four sequence kinds; `S[N]` is that kind with bound `N`; `elem(S)` is its element type (2.3.1). All sequence primitives are kind-preserving: a `concat` of two buffers is a buffer.

Primitive	Signature	Semantics	Runtime errors
`len`	`(S[N]) -> uint`	current length (code points for `string-utf8`)	
`concat`	`(S[N] S[M]...) -> S[N + M1 + ... + Mk]`	concatenation, left to right; with rest arguments of bounds `M1 ... Mk`, the bound is their exact sum plus `N`. With a spread `(list K S[M])`, the bound is `N + K × M`	
`element-at?`	`(S[N] uint) -> (optional elem(S))`	element at zero-based index, or `none` if out of range	

Primitive	Signature	Semantics	Runtime errors
`slice?`	`(S[N] uint uint) -> (optional S[N])`	elements in `[left, right)`; `none` if `left > right` or `right > len`	
`as-max-len?`	`(S[N] size) -> (optional S[M])`	the second argument is a size term `M` (literal or size variable); `(some s)` if `len(s) <= M`, else `none`	
`map`	`((function (elem(S)) U) S[N]) -> (list N U)`	applies the function to each element in order	
`filter`	`((function (elem(S)) bool) S[N]) -> S[N]`	keeps elements for which the function returns `true`, preserving order	
`fold-left`	`((function (A elem(S)) A) A S[N]) -> A`	`(f (f (f init x1) x2) ... xn)`: elements left to right, accumulator first	
`fold-right`	`((function (elem(S) A) A) S[N] A) -> A`	`(f x1 (f x2 ... (f xn init)))`: elements right to left, accumulator second	

In both folds the initial value sits on the same side as the accumulator in the callback, so the argument order of the call mirrors the order of the callback's parameters. Neither fold can be written from the other: that needs a wrapper that swaps `f`'s arguments, and a function cannot be closed over. Legacy `fold` (left to right, element first) is removed; it is `fold-left` with the callback's parameters swapped.

Library, not primitive: `append`, `index-of?`, `replace-at?`. Legacy multi-sequence `map` (`(map f l1 l2)`) is dropped. It cannot be written in the library, because a fold would have to carry `f` in its accumulator; if it is needed it can return as a primitive family (`map2`, `map3`).

3.4 Conversions and serialization

Primitive	Signature	Semantics	Runtime errors
`buff-to-uint-be`	`((buff 16)) -> uint`	big-endian unsigned interpretation; the empty buffer is `u0`	
`uint-to-buff-be`	`(uint size) -> (buff W)`	the second argument is a size term `W` (literal or static constant), at most 16; big-endian encoding in exactly `W` bytes	value does not fit in `W` bytes
`uint-to-buff-le`	`(uint size) -> (buff W)`	little-endian counterpart	value does not fit
`int-to-buff-be`	`(int size) -> (buff W)`	two's-complement big-endian encoding in exactly `W` bytes	value does not fit in `W` signed bytes

Primitive	Signature	Semantics	Runtime errors
`int-to-buff-le`	`(int size) -> (buff W)`	little-endian counterpart	value does not fit
`to-consensus-buff`	`(T) -> (buff serialized-size(T))`	SIP-005 serialization of the value. Total: the size limits apply to the result through its position (2.3.1), so a serialization that is only hashed or compared is bounded by the intermediate limit, and one that is bound, returned, or stored by the value limit, both statically. `T` MUST be storable	
`from-consensus-buff?`	`(type (buff N)) -> (optional T)`	the first argument is a type expression `T`, which MUST be storable. Deserializes; `none` if the bytes are not a canonical encoding of a value of type `T` (exact typed-tuple fields, no trailing bytes). Memory for the result is reserved against `serialized-size(T)` before decoding (2.7.7)	

`serialized-size(T)` is a type-level size function defined with the wire format in the chain ABI.

`buff-to-uint-be` and `uint-to-buff-be` are the base conversions between integers and bytes. The `*-to-buff-*` family stays primitive because each takes a size argument, which a library function cannot. Library, not primitive: `buff-to-uint-le`, `buff-to-int-be`, `buff-to-int-le`, `int-to-ascii`, `int-to-utf8`, `string-to-int?`, `string-to-uint?`, and `to-ascii?`, with the same signatures as legacy. All but the signed buffer conversions also have the same results. `buff-to-int-be` and `buff-to-int-le` sign-extend from the buffer's actual length, so `(buff-to-int-be 0xff)` is `-1`; legacy zero-pads a short buffer and returns `255`, which makes the signed conversion of a short buffer disagree with the same bytes in a 16-byte buffer. This is a deliberate change, listed in the footguns table of chapter 1 and the disposition table of chapter 4.

The four `*-to-buff-*` primitives have no legacy counterpart. They are the write direction of the `buff-to-*` family, requested for hashing structured data (Merkle leaves, transcripts, commitments); legacy contracts slice the type-tagged output of `to-consensus-buff` instead, which is fragile and aliases values that share low bytes. Rejecting values that do not fit closes that aliasing class at the language level.

3.5 Principals

Primitive	Signature	Semantics
`principal-construct?`	`((buff 1) (buff 20) (optional (string-ascii 40))) -> (response principal { error-code: uint, principal: (optional principal) })`	builds a standard principal (no name) or contract principal (with name) from a version byte and hash bytes. `(err {error-code: u0, principal: (some p)})` if the version byte is valid but for the other network; `u1` if the version byte is invalid; `u2` if the name is not a valid contract name

Primitive	Signature	Semantics
`principal-destruct?`	`(principal) -> (response D D)` where `D = { hash-bytes: (buff 20), name: (optional (string-ascii 40)), version: (buff 1) }`	decomposes a principal. `ok` if the version byte matches the current network, `err` with the same payload otherwise
`principal-to-ascii`	`(principal) -> (string-ascii 82)`	the principal's text form: the c32check address of a standard principal, or the address, `.`, and the name of a contract principal (41 for the address, 1 for the dot, 40 for the name)
`contract-of`	`((trait X)) -> principal`	the contract principal a trait-typed value refers to
`contract-hash?`	`(principal) -> (response (buff 32) uint)`	SHA-512/256 of the deployed contract's module; `err u1` if the principal is not a deployed contract. The chain never holds source; the source hash is available from the module's provenance section, off chain

Library, not primitive: `is-current-network`, `is-standard`, `principal-of?`.

`principal-to-ascii` is new. It is the principal case of `to-ascii?`, which is otherwise library (3.4). It stays in the host because c32check is a chain address encoding that wallets and explorers must match byte for byte.

3.6 Cryptography

Hash functions accept buffers only (P6, footguns table). Library, not primitive: `verify-merkle-proof`, a composition of `sha256` (chapter 4).

Primitive	Signature	Semantics
`hash160`	`((buff N)) -> (buff 20)`	RIPEMD-160 of SHA-256
`sha256`	`((buff N)) -> (buff 32)`	SHA-256
`sha512`	`((buff N)) -> (buff 64)`	SHA-512
`sha512/256`	`((buff N)) -> (buff 32)`	SHA-512/256 (distinct IV; not a truncation of SHA-512)
`keccak256`	`((buff N)) -> (buff 32)`	Keccak-256
`secp256k1-recover?`	`((buff 32) (buff 65)) -> (response (buff 33) uint)`	recovers the compressed public key from a message hash and a recoverable signature; `err u1` malformed or non-canonical signature, `err u2` recovery failed

Primitive	Signature	Semantics
`secp256k1-verify`	`((buff 32) (buff 65) (buff 33)) -> bool`	verifies a signature (64 bytes, or 65 with the recovery byte ignored) against a compressed public key; `false` for a non-canonical signature
`secp256k1-decompress?`	`((buff 33)) -> (response (buff 65) uint)`	uncompressed form of a compressed public key
`secp256r1-verify`	`((buff 32) (buff 64) (buff 33)) -> bool`	P-256 signature verification; `false` for a non-canonical signature
`ed25519-verify`	`((buff N) (buff 64) (buff 32)) -> bool`	Ed25519 signature verification over an arbitrary-length message

Canonical signatures. An ECDSA signature (secp256k1 or P-256) is canonical only if its `S` component is in the lower half of the curve order (low-S). The verify primitives MUST return `false`, and `secp256k1-recover?` MUST return `err u1`, for a non-canonical signature. Accepting both `S` and `n - S` would let anyone replay a signature-gated authorization with a different signature encoding.

3.7 Contract storage

Primitive	Signature	Semantics
`var-get`	`((var T)) -> T`	current value
`var-set`	`((var T) T) -> bool`	stores the value; returns `true`
`map-get?`	`((map K V) K) -> (optional V)`	lookup
`map-set`	`((map K V) K V) -> bool`	inserts or overwrites; returns `true`
`map-delete`	`((map K V) K) -> bool`	removes the entry; returns `true` if it existed

Library, not primitive: `map-insert?` (`map-get?` then `map-set`). A membership test is `(is-some (map-get? m k))`; a compiler MAY lower it to an existence check that does not read the value, since the result is identical (P5).

3.8 Assets

All asset primitives record movements in the transaction's asset map (for post-conditions and allowances) and emit events.

Primitive	Signature	Semantics	`err` codes
`ft-mint?`	`((fungible-token) uint principal) -> (response bool uint)`	mints `amount` to the recipient; exceeding the supply cap is a runtime error, as in legacy	`u1` amount is zero

Primitive	Signature	Semantics	`err` codes
`ft-burn?`	`((fungible-token) uint principal) -> (response bool uint)`	burns `amount` from the owner	`u1` insufficient balance or amount is zero
`ft-transfer?`	`((fungible-token) uint principal principal) -> (response bool uint)`	moves `amount` from sender to recipient	`u1` insufficient balance; `u2` sender is recipient; `u3` amount is zero
`ft-get-balance`	`((fungible-token) principal) -> uint`	balance	
`ft-get-supply`	`((fungible-token)) -> uint`	circulating supply	
`nft-mint?`	`((non-fungible-token T) T principal) -> (response bool uint)`	mints the identifier to the recipient	`u1` already exists
`nft-burn?`	`((non-fungible-token T) T principal) -> (response bool uint)`	burns the identifier owned by the owner	`u1` not owned by owner; `u3` does not exist
`nft-transfer?`	`((non-fungible-token T) T principal principal) -> (response bool uint)`	changes the owner	`u1` not owned by sender; `u2` sender is recipient; `u3` does not exist
`nft-get-owner?`	`((non-fungible-token T) T) -> (optional principal)`	owner	
`stx-transfer?`	`(uint principal principal (optional (buff 34))) -> (response bool uint)`	moves uSTX with an optional memo (legacy `stx-transfer?` and `stx-transfer-memo?` merged)	`u1` insufficient balance; `u2` sender is recipient; `u3` amount is zero; `u4` sender is not the context's `sender`
`stx-burn?`	`(uint principal) -> (response bool uint)`	burns uSTX	`u1` insufficient balance; `u3` amount is zero; `u4` sender is not the context's `sender`
`stx-account`	`(principal) -> { locked: uint, unlock-height: uint, unlocked: uint }`	STX account state	

`stx-transfer?` and `stx-burn?` require the `sender` argument to be the context's `sender` field (3.9), returning `err u4` otherwise. The `ft-*` and `nft-*` primitives do not check the sender; every movement is recorded in the asset map and is subject to transaction post-conditions and to any enclosing `as-contract?` or `restrict-assets?` allowance frame. Open question: whether vNEXT should require the context's `sender` for token transfers as well.

Library, not primitive: `stx-get-balance`.

3.9 Transaction and chain context

The execution context is one record, read with a single primitive. Legacy's twelve bare context names (`tx-sender`, `stacks-block-height`, and so on) are its fields; `(get sender (get-current-context))` replaces `tx-sender`.

Primitive	Signature	Semantics
`get-current-context`	`() -> context`	the context of the current call frame; the fields are below. A compiler MAY lower `(get f (get-current-context))` to a read of that field alone

```
context = { sender: principal, caller: principal, sponsor: (optional principal),
            contract: principal,
            stacks-height: uint, burn-height: uint, tenure-height: uint,
            time: uint, liquid-supply: uint,
            chain-id: uint, is-mainnet: bool, is-regtest: bool }
```

Field	Value
`sender`	the transaction sender, or the current contract inside `as-contract?`
`caller`	the principal that made the current call: the sender for the entry call, the calling contract for `contract-call?`, the current contract inside `as-contract?`
`sponsor`	the sponsor, if the transaction is sponsored
`contract`	the contract being executed
`stacks-height`	height of the current Stacks block
`burn-height`	height of the burnchain block the current Stacks block is anchored to
`tenure-height`	number of tenures elapsed
`time`	timestamp of the current Stacks block (seconds)
`liquid-supply`	total liquid uSTX
`chain-id`	chain identifier
`is-mainnet`	true on mainnet
`is-regtest`	true in regtest environments

`sender`, `caller`, and `contract` are frame-scoped: `as-contract?` (2.5.9) and `contract-call?` change them for the frames they create. The other fields are the same in every frame of a transaction.

Context and chain data are records rather than properties selected by name, so their typing is uniform and there is no property-name mini-language.

Lookups are total. Every chain-data lookup returns `none` for any height or input the host cannot serve, including heights before the chain's first block and data that has not yet matured. A chain-data lookup MUST NOT raise a runtime error and MUST NOT make the transaction unmineable. (Legacy treated some such lookups as "problematic" transactions that miners drop, which desynchronizes the sender's nonce.)

Primitive	Signature	Semantics
`get-stacks-block-info?`	`(uint) -> (optional { id-header-hash: (buff 32), header-hash: (buff 32), time: uint })`	`none` if the height is not less than the current height: the current block's hashes do not exist until it is sealed, after this transaction runs. The current block's time is the context's `time` field
`get-tenure-info?`	`(uint) -> (optional { burnchain-header-hash: (buff 32), miner-address: principal, time: uint, vrf-seed: (buff 32), block-reward: (optional uint), miner-spend-total: uint, miner-spend-winner: uint })`	serves every tenure up to and including the current one (the context's `tenure-height`), since all fields are fixed at sortition. `block-reward` is `none` until the reward has matured. Maturity is defined by the host in terms of the coinbase (tenure) height at which the reward was recorded, so the lookup is total: it never fails for a height the host has no row for
`get-burn-block-info?`	`(uint) -> (optional { header-hash: (buff 32), pox-addrs: { addrs: (list 2 { hash-bytes: (buff 32), version: (buff 1) }), payout: uint } })`	serves every burn block up to and including the current one (the context's `burn-height`); `none` if the burn height is outside the known range. The `hashbytes` field of legacy is `hash-bytes` (P6)
`get-bitcoin-tx-output?`	`((buff N) uint) -> (response { script: (buff 1024), amount: uint, txid: (buff 32) } uint)`	parses raw Bitcoin transaction bytes (SegWit or not) and returns output `vout`, with `txid` in internal byte order; `err u1` the bytes do not deserialize as a transaction, `err u2` `vout` is out of range, `err u3` the output script exceeds 1024 bytes

3.10 Events and control

Primitive	Signature	Semantics
`print`	`(T) -> T`	emits the value as an event and returns it
`panic`	`() -> never`	aborts the transaction with a runtime error. The library's `unwrap-panic` and `unwrap-err-panic` are defined with it

3.11 Count

Group	Primitives
Arithmetic	11 (each in two integer families)

Group	Primitives
Comparison and equality	5
Sequences	9
Conversions	8
Principals	5
Cryptography	10
Storage	5
Assets	12
Context	5
Events and control	2
Total	**72**

For comparison, legacy Clarity 6 has 126 native functions and 16 native variables, most with bespoke typing rules.

4. Standard Library

The standard library is vNEXT source, linked into modules by the compiler as a prelude that is in scope in every contract (2.4.5). Its normative definition is its source: std/prelude.clar. This chapter explains how it is delivered, what it contains and why, and how every legacy Clarity construct maps onto vNEXT.

4.1 Delivery and status

- The prelude is part of the language version. It is frozen when the language version is frozen and never changes for that version.
- Prelude names are reserved: a contract MUST NOT define a top-level name that the prelude defines. This includes the `std-*` helper definitions.
- The prelude uses only features available to every contract (P3). It is type-checked, instantiated, and compiled by the same rules as user code.
- **Results are defined by the source.** Conformance vectors for library functions are derived from the prelude source. If a vector and the source disagree, the vector is wrong.
- **Linked into each module.** The compiler includes the library instances a contract uses in that contract's module. A library call costs the fuel of that code (P4, P5). The chain holds no library code of its own.
- **Optimization replaces the implementation, not the definition.** A later compiler MAY link a hand-written or better compiled Wasm implementation of a library function. It MUST produce identical results, events, and errors to the source for every input, which the conformance suite checks. Contracts already deployed keep the code they were deployed with, so a library change never alters them and no chain upgrade is involved. (Their cost can still change at an epoch boundary if the chain's fuel schedule does; see ../03-architecture.md section 4.)

The source is written for correctness and readability, not speed. Where a definition is slow (a digit-by-digit conversion, a bitwise square root), that is a candidate for a Wasm replacement, not a reason to move it into the core.

4.2 What belongs in the library

Everything that can be written in the language. The layering rule (README.md) keeps in the core only host access, what cannot be expressed, cryptographic standards, and the base operations on each type's representation. Every other construct a contract author uses is here.

Section	Definitions	Built from
Branching	`if`, `not`	`match` on `bool`

Section	Definitions	Built from
Early exit	`asserts!`, `unwrap!`, `unwrap-err!`, `try!`, `unwrap-panic`, `unwrap-err-panic`, `default-to`	`match`, `return`, `panic`
Predicates	`is-some`, `is-none`, `is-ok`, `is-err`	`match`
Arithmetic	`max`, `min` (rest parameters), `abs`, `mod`, `bit-not`	comparison, `+ - * /`, `bit-xor`, `fold-left`
Arithmetic	`pow`, `sqrti`, `log2`	`fold-left` over a constant list of bit positions, with shifts
Sequences	`append`, `replace-at?`	`concat`, `slice?`, `as-max-len?`
Sequences	`index-of?`	`fold-left` with the target in the accumulator
Conversions	`buff-to-uint-le`, `buff-to-int-be`, `buff-to-int-le`	`buff-to-uint-be`, a byte-reversing `fold-left`, two's-complement arithmetic sign-extended from the actual length
Conversions	`int-to-ascii`, `int-to-utf8`, `string-to-int?`, `string-to-uint?`	digit tables, `element-at?`, `index-of?`, a 39-step `fold-left`
Conversions	`to-ascii?`	`int-to-ascii`, `principal-to-ascii`, `map` with a hex or character table, `concat` with a spread
Principals	`is-current-network`, `is-standard`, `principal-of?`	`principal-destruct?`, `principal-construct?`, `hash160`
Cryptography	`verify-merkle-proof`	`sha256`, `log2`, `fold-left` over the proof
Storage	`map-insert?`	`map-get?`, `map-set`
Assets	`stx-get-balance`	`stx-account`

About forty public definitions, plus `std-*` helpers. Two conventions decide whether a prelude function is `define-inline` or `define-private`:

- Anything that needs laziness or early exit is inline: `if` and the early-exit family.
- Everything else is private. Private functions are compiled once per instance and can be passed to `map`, `filter`, `fold-left`, and `fold-right`.

Notes on specific definitions

- **Families over integer or sequence kinds.** `pow`, `sqrti`, `log2`, `bit-not`, `int-to-ascii`, and `int-to-utf8` are overloaded on `int` and `uint`. `replace-at?` is overloaded on the four sequence kinds. `mod`, `max`, `min`, `index-of?`, `string-to-int?`, and `string-to-uint?` are single generic definitions checked per instance (2.6.2); per-kind constants such as digit tables are selected by small overloaded helpers.
- **`pow` does not overflow early.** It squares and multiplies from the most significant exponent bit down, so every intermediate is the base raised to a prefix of the exponent and no step overflows

unless the result does.

- `verify-merkle-proof` takes a leaf hash, a root hash, the leaf index, the transaction count, and up to 24 sibling hashes from leaf to root, all in internal byte order, and uses double SHA-256 with Bitcoin's odd-row duplication rule. It returns `false` for any proof inconsistent with the canonical tree shape implied by the count: an index not below the count, a path length other than the tree depth, or a sibling that must equal its partner on an odd row but does not. This contract is enforced by the source; it is security-sensitive and needs the densest conformance vectors in the library.

Kept primitive although expressible

The layering rule's representation-base test keeps a few operations that could be written in the language, because everything else is written in terms of them: `element-at?` (expressible through `slice?`), `to-int` and `to-uint` (expressible through the byte conversions), the bitwise operations, and comparison on sequences. Moving them would lengthen the prelude without shrinking the set of things a compiler must implement.

`principal-to-ascii` is also expressible (c32check is base-32 text with a double-SHA-256 checksum) and is kept primitive for a different reason: c32check is the chain's address encoding, defined and maintained with the chain, so the host renders it rather than the language fixing it in a frozen prelude.

4.3 Disposition of every legacy construct

This table covers every native function, define form, and native variable in Clarity 6. "Primitive" means it stays in the core with the signature in 03-primitives.md; "form" means a special form in 02-core-language.md; "library" means it is defined in the prelude; "removed" means vNEXT has no equivalent by that name.

Arithmetic and logic

Legacy	vNEXT	Notes
`+`, `-`, `*`, `/`	primitive	rest parameter (2.6.5); at least two arguments, so unary `-` is written `(- 0 x)`
`mod`, `pow`, `sqrti`, `log2`	library	same results as legacy
`>=`, `<=`, `<`, `>`	primitive	
`to-int`, `to-uint`	primitive	
`xor`	removed	use `bit-xor`
`bit-and`, `bit-or`, `bit-xor`	primitive	rest parameter
`bit-not`	library	`bit-xor` with all ones
`bit-shift-left`, `bit-shift-right`	primitive	shift of 128 or more aborts instead of wrapping

Legacy	vNEXT	Notes
`and`, `or`	form	two or more arguments, short-circuit
`not`	library	private
`is-eq`	primitive	binary only; `(is-eq a b c)` is written `(and (is-eq a b) (is-eq a c))`
`if`	library	inline over `match`

Binding, sequencing, branching, early exit

Legacy	vNEXT	Notes
`let`	form	
`begin`	sugar	`(let () ...)`
`match`	form	constructor patterns `((some x) ...)`, `(none ...)`, `((ok x) ...)`, `((err e) ...)`, `(true ...)`, `(false ...)`
`asserts!`, `unwrap!`, `unwrap-err!`, `try!`	library	inline over `match` and `return`; `thrown` is evaluated only on failure, which for `unwrap!` and `unwrap-err!` is a change from legacy (legacy evaluates it first); not usable in top-level expressions or constant initializers, where `return` is forbidden (2.4.2)
`unwrap-panic`, `unwrap-err-panic`	library	inline over `match` and `panic`
`default-to`	library	private; both arguments evaluated, as in legacy
`is-ok`, `is-err`, `is-some`, `is-none`	library	private
`ok`, `err`, `some`	form	constructors
`none`, `true`, `false`	literal	

Sequences

Legacy	vNEXT	Notes
`list`	form	variadic, like `and` and `or`; functions use rest parameters
`len`, `concat`, `slice?`, `as-max-len?`	primitive	`concat` has a rest parameter
`element-at?`	primitive	
`index-of?`, `replace-at?`	library	
`element-at`, `index-of`	removed	aliases
`append`	library	`(concat items (list item))`

Legacy	vNEXT	Notes
`map`	primitive	single sequence only; multi-sequence form dropped (chapter 3.3)
`filter`	primitive	
`fold`	removed	`fold-left` with the callback's parameters swapped (accumulator first)
new: `fold-left`, `fold-right`	primitive	3.3

Tuples

Legacy	vNEXT	Notes
`tuple`	removed	use `{ f: v, ... }`
`get`, `merge`	form	

Conversions and serialization

Legacy	vNEXT	Notes
`buff-to-uint-be`	primitive	
`buff-to-uint-le`	library	over `buff-to-uint-be`
`buff-to-int-be`, `buff-to-int-le`	library	over `buff-to-uint-be`; a short buffer is sign-extended from its actual length, where legacy zero-padded it (chapter 3.4)
new: `uint-to-buff-be`, `uint-to-buff-le`, `int-to-buff-be`, `int-to-buff-le`	primitive	fixed width, abort if the value does not fit (issue #7312)
`int-to-ascii`, `int-to-utf8`	library	
`string-to-int?`, `string-to-uint?`	library	
`to-consensus-buff?`	primitive	renamed `to-consensus-buff`, total: legacy's `none` for a result over the value size limit is replaced by the static size limits on the result's position (2.3.1)
`from-consensus-buff?`	primitive	

Legacy	vNEXT	Notes
`to-ascii?`	library	six overloads (`int`, `uint`, `bool`, `principal`, `buff`, `string-utf8`) with legacy's text and bounds, except that the principal bound is 82 where legacy's is 170, because contract names are limited to 40 characters (2.2); the principal case uses the new `principal-to-ascii` primitive (chapter 3.5). A `string-utf8` containing U+000C (form feed) is now `err u1`: legacy accepted it, but a vNEXT `string-ascii` cannot hold it (2.2)

Principals

Legacy	vNEXT	Notes
`principal-construct?`	primitive	single signature with `(optional (string-ascii 40))` name; `error_code` field renamed `error-code`
`principal-destruct?`	primitive	
`is-standard`	library	renamed `is-current-network`, since it tests the version byte against the current network and is true for contract principals too; vNEXT's `is-standard` tests for a standard (not contract) principal
`principal-of?`	library	`hash160` plus `principal-construct?` with the network's version byte
`contract-of`	primitive	
`contract-hash?`	primitive	

Cryptography

Legacy	vNEXT	Notes
`hash160`, `sha256`, `sha512`, `sha512/256`, `keccak256`	primitive	buffers only; integer inputs removed
`secp256k1-recover?`, `secp256k1-verify`, `secp256k1-decompress?`	primitive	low-S required
`secp256r1-verify`, `ed25519-verify`	primitive	low-S required for ECDSA
`verify-merkle-proof`	library	over `sha256`

Storage

Legacy	vNEXT	Notes
`var-get`, `var-set`	primitive	

Legacy	vNEXT	Notes
`map-get?`, `map-set`, `map-delete`	primitive	
`map-insert`	library	renamed `map-insert?`; returns `(optional V)`: the existing value if the key was present (nothing written), `none` if inserted, where legacy returned `bool`; private, so both arguments are evaluated; written with a `(map K V)` reference parameter (2.3.6)

Assets

Legacy	vNEXT	Notes
`ft-mint?`, `ft-burn?`, `ft-transfer?`, `ft-get-balance`, `ft-get-supply`	primitive	
`nft-mint?`, `nft-burn?`, `nft-transfer?`, `nft-get-owner?`	primitive	
`stx-transfer?`	primitive	takes `(optional (buff 34))` memo
`stx-transfer-memo?`	removed	merged into `stx-transfer?`
`stx-burn?`, `stx-account`	primitive	
`stx-get-balance`	library	`(get unlocked (stx-account p))`

Calls, context, and allowances

Legacy	vNEXT	Notes
`contract-call?`	form	
`as-contract?`	form	
`as-contract`	removed	unrestricted asset access
`restrict-assets?`	form	`with-all-assets-unsafe` not permitted
`with-stx`, `with-ft`, `with-nft`, `with-staking`, `with-pox`, `with-all-assets-unsafe`	allowance forms	
`with-stacking`	removed	renamed `with-staking` in Clarity 6
`at-block`	removed	disabled in legacy since epoch 3.4
`print`	primitive	

Chain data

Legacy	vNEXT	Notes
`get-stacks-block-info?`, `get-tenure-info?`, `get-burn-block-info?`	primitive	return the whole record; property-name argument removed; `get-tenure-info?` and `get-burn-block-info?` serve the current tenure and burn block; the `hashbytes` field is `hash-bytes`
`get-block-info?`	removed	already removed in Clarity 3
`get-bitcoin-tx-output?`	primitive	

Define forms

Legacy	vNEXT	Notes
`define-constant`, `define-data-var`, `define-map`	form	
`define-fungible-token`, `define-non-fungible-token`	form	
`define-public`, `define-read-only`, `define-private`	form	private may be generic and overloaded
`define-trait`, `impl-trait`	form	trait identifiers replace `<name>` syntax
`use-trait`	removed	`(define-type name (trait id))`
new: `define-type`	form	transparent type aliases
new: `define-inline`	form	call-by-name, expanded at call site
new: rest parameters `(name T...)`	form	private functions only; the rest arguments are bound as a list (2.6.5)

Native variables

Legacy	vNEXT	Notes
`tx-sender`, `contract-caller`, `tx-sponsor?`, `current-contract`	removed	fields `sender`, `caller`, `sponsor`, `contract` of `(get-current-context)` (3.9); `(get sender (get-current-context))` replaces `tx-sender`
`burn-block-height`, `stacks-block-height`, `tenure-height`, `stacks-block-time`	removed	fields `burn-height`, `stacks-height`, `tenure-height`, `time` of `(get-current-context)`
`chain-id`, `is-in-mainnet`, `is-in-regtest`, `stx-liquid-supply`	removed	fields `chain-id`, `is-mainnet`, `is-regtest`, `liquid-supply` of `(get-current-context)`
new: `get-current-context`	primitive	one record for the whole execution context; the twelve bare names are no longer reserved
`block-height`	removed	already removed in Clarity 3

Legacy	vNEXT	Notes
`none`, `true`, `false`	literal	

4.4 New in vNEXT

Library additions with no legacy counterpart, included because they were common user-written helpers:

- `max`, `min` (rest parameters), `abs`

Primitive additions with no legacy counterpart: the four `*-to-buff-*` functions and `principal-to-ascii` (chapter 3). Further candidates from the issue tracker are collected in ../06-issue-tracker.md.

Appendix: `spec/std/prelude.clar`

```
;; Clarity vNEXT standard library prelude (working draft)
;;
;; This file is ordinary vNEXT source. It uses only features available to
;; every contract: generics (implicit type variables), inline functions with
;; call-by-name parameters, overloading, static references, `match`,
;; `return`, and the primitives in ../03-primitives.md.
;;
;; Scope: everything a contract author uses that can be written in the
;; language (layering rule, README.md). Definitions are written for clarity
;; and correctness, not speed. Cost is the Wasm fuel of the compiled code, so
;; a faster Wasm implementation can replace any definition later without
;; changing its results (chapter 4.1).
;;
;; Conventions:
;;   define-inline  for anything needing laziness or early exit
;;   define-private for everything else
;;   fold-left      calls (step accumulator element), so helpers take acc first
;;   (name T...)    a rest parameter: the extra arguments as a list (2.6.5)
;;   std-*          helper definitions; reserved like every prelude name
;;
;; Generic bodies are checked per instance (template semantics, chapter
;; 2.6.2). Where a comment says "`T` is instantiated at ...", any other
;; instantiation is a type error at the call site.
;;
;; Each value parameter of an inline function is referenced at most once in
;; its body; bodies that need a value twice bind it with `let` first.

;; ---------------------------------------------------------------------------
;; Branching
;; ---------------------------------------------------------------------------

;; `if` is sugar over `match` on bool. `and` and `or` are core forms, since
;; short-circuiting over any number of arguments cannot be written here
;; (chapter 2.5.3).
(define-inline (if (condition bool) (then T) (else T))
  (match condition
    (true then)
    (false else)))

;; Private so it can be passed to `filter`.
(define-private (not (value bool))
  (if value false true))

;; ---------------------------------------------------------------------------
;; Early exit
;; ---------------------------------------------------------------------------

(define-inline (asserts! (condition bool) (thrown E))
  (if condition true (return thrown)))

;; `thrown` is evaluated only on failure. For `unwrap!` and `unwrap-err!`
;; this is a change from legacy, which evaluated it before the check;
;; `asserts!` was already lazy.
(define-inline (unwrap! (value (optional T)) (thrown E))
  (match value
    ((some inner) inner)
    (none (return thrown))))

(define-inline (unwrap! (value (response T E)) (thrown F))
  (match value
    ((ok inner) inner)
```

```
    ((err ignored) (return thrown))))

(define-inline (unwrap-err! (value (response T E)) (thrown F))
  (match value
    ((ok ignored) (return thrown))
    ((err inner) inner)))

(define-inline (try! (value (optional T)))
  (match value
    ((some inner) inner)
    (none (return none))))

(define-inline (try! (value (response T E)))
  (match value
    ((ok inner) inner)
    ((err inner) (return (err inner)))))

(define-inline (unwrap-panic (value (optional T)))
  (match value
    ((some inner) inner)
    (none (panic))))

(define-inline (unwrap-panic (value (response T E)))
  (match value
    ((ok inner) inner)
    ((err ignored) (panic))))

(define-inline (unwrap-err-panic (value (response T E)))
  (match value
    ((ok ignored) (panic))
    ((err inner) inner)))

;; Private, so both arguments are evaluated before the call, as in legacy.
(define-private (default-to (default T) (value (optional T)))
  (match value
    ((some inner) inner)
    (none default)))

;; ---------------------------------------------------------------------------
;; Predicates (private so they work with `filter`)
;; ---------------------------------------------------------------------------

(define-private (is-some (value (optional T)))
  (match value ((some ignored) true) (none false)))

(define-private (is-none (value (optional T)))
  (match value ((some ignored) false) (none true)))

(define-private (is-ok (value (response T E)))
  (match value ((ok ignored) true) ((err ignored) false)))

(define-private (is-err (value (response T E)))
  (match value ((ok ignored) false) ((err ignored) true)))

;; ---------------------------------------------------------------------------
;; Arithmetic
;; ---------------------------------------------------------------------------

;; `T` is instantiated at int or uint. (max a b c) folds the binary helper
;; over (list b c) starting from a.
(define-private (std-max-pair (left T) (right T))
  (if (> left right) left right))

(define-private (max (left T) (right T...))
  (fold-left std-max-pair left right))

(define-private (std-min-pair (left T) (right T))
```

```
  (if (< left right) left right))

(define-private (min (left T) (right T...))
  (fold-left std-min-pair left right))

;; Aborts on the most negative int, as `(- 0 x)` overflows there.
(define-private (abs (value int))
  (if (< value 0) (- 0 value) value))

;; `T` is instantiated at int or uint. Division truncates toward zero, so the
;; remainder takes the sign of the dividend. A zero divisor aborts in `/`, as
;; does the most negative int with divisor -1, although its remainder is 0.
(define-private (mod (dividend T) (divisor T))
  (- dividend (* (/ dividend divisor) divisor)))

(define-private (bit-not (value int))
  (bit-xor value -1))

(define-private (bit-not (value uint))
  (bit-xor value u340282366920938463463374607431768211455))

;; Exponent bit positions, most significant first.
(define-constant std-exponent-bits
  (list u31 u30 u29 u28 u27 u26 u25 u24 u23 u22 u21 u20 u19 u18 u17 u16
        u15 u14 u13 u12 u11 u10 u9 u8 u7 u6 u5 u4 u3 u2 u1 u0))

;; Square-and-multiply, most significant bit first. Every intermediate value
;; is `base` raised to a prefix of the exponent's bits, so no step overflows
;; unless the final result does. `T` is int or uint.
(define-private (std-pow-step (acc { base: T, exponent: uint, result: T })
                              (shift uint))
  (let ((squared (* (get result acc) (get result acc))))
    (merge acc
      { result: (if (is-eq (bit-and (bit-shift-right (get exponent acc) shift) u1) u1)
                  (* squared (get base acc))
                  squared) })))

;; Aborts if the exponent is negative or does not fit in 32 bits, or on
;; overflow.
(define-private (pow (base int) (exponent int))
  (if (or (< exponent 0) (> exponent 4294967295))
    (panic)
    (get result (fold-left std-pow-step
                  { base: base, exponent: (to-uint exponent), result: 1 }
                  std-exponent-bits))))

(define-private (pow (base uint) (exponent uint))
  (if (> exponent u4294967295)
    (panic)
    (get result (fold-left std-pow-step
                  { base: base, exponent: exponent, result: u1 }
                  std-exponent-bits))))

;; Even bit positions from 126 down to 0: the powers of four tried by the
;; bitwise square root.
(define-constant std-sqrti-shifts
  (list u126 u124 u122 u120 u118 u116 u114 u112 u110 u108 u106 u104 u102
        u100 u98 u96 u94 u92 u90 u88 u86 u84 u82 u80 u78 u76 u74 u72 u70
        u68 u66 u64 u62 u60 u58 u56 u54 u52 u50 u48 u46 u44 u42 u40 u38
        u36 u34 u32 u30 u28 u26 u24 u22 u20 u18 u16 u14 u12 u10 u8 u6 u4
        u2 u0))

(define-private (std-sqrti-step (acc { remainder: uint, root: uint })
                                (shift uint))
  (let ((bit (bit-shift-left u1 shift))
        (candidate (+ (get root acc) bit)))
    (if (>= (get remainder acc) candidate)
```

```
      { remainder: (- (get remainder acc) candidate),
        root: (+ (bit-shift-right (get root acc) u1) bit) }
      { remainder: (get remainder acc),
        root: (bit-shift-right (get root acc) u1) }))

(define-private (std-sqrti-uint (value uint))
  (get root (fold-left std-sqrti-step { remainder: value, root: u0 }
              std-sqrti-shifts)))

(define-private (sqrti (value uint))
  (std-sqrti-uint value))

;; Aborts on a negative argument.
(define-private (sqrti (value int))
  (if (< value 0)
    (panic)
    (to-int (std-sqrti-uint (to-uint value)))))

;; Binary search for the highest set bit.
(define-private (std-log2-step (acc { rest: uint, result: uint }) (shift uint))
  (let ((shifted (bit-shift-right (get rest acc) shift)))
    (if (is-eq shifted u0)
      acc
      { rest: shifted, result: (+ (get result acc) shift) })))

;; Aborts on zero.
(define-private (std-log2-uint (value uint))
  (if (is-eq value u0)
    (panic)
    (get result (fold-left std-log2-step { rest: value, result: u0 }
                  (list u64 u32 u16 u8 u4 u2 u1)))))

(define-private (log2 (value uint))
  (std-log2-uint value))

;; Aborts on an argument that is not positive.
(define-private (log2 (value int))
  (if (<= value 0)
    (panic)
    (to-int (std-log2-uint (to-uint value)))))

;; ---------------------------------------------------------------------------
;; Sequences
;; ---------------------------------------------------------------------------

;; (list N T) -> (list (+ N 1) T)
(define-private (append (items (list N T)) (item T))
  (concat items (list item)))

(define-private (std-index-of-step (acc { found: (optional uint), index: uint, target: T })
                                   (element T))
  (match (get found acc)
    ((some ignored) acc)
    (none (if (is-eq element (get target acc))
            (merge acc { found: (some (get index acc)) })
            (merge acc { index: (+ (get index acc) u1) })))))

;; `S` is instantiated at a sequence type and `E` at its element type.
(define-private (index-of? (items S) (item E))
  (get found (fold-left std-index-of-step
               { found: none, index: u0, target: item }
               items)))

;; One definition per sequence kind: a list takes `(list item)` where the
;; other kinds take the one-element sequence itself.
(define-private (replace-at? (items (list N T)) (index uint) (item T))
  (if (>= index (len items))
```

```
    none
    (as-max-len? (concat (concat (unwrap-panic (slice? items u0 index)) (list item))
                         (unwrap-panic (slice? items (+ index u1) (len items))))
                 N)))

(define-private (replace-at? (items (buff N)) (index uint) (item (buff 1)))
  (if (>= index (len items))
    none
    (as-max-len? (concat (concat (unwrap-panic (slice? items u0 index)) item)
                         (unwrap-panic (slice? items (+ index u1) (len items))))
                 N)))

(define-private (replace-at? (items (string-ascii N)) (index uint) (item (string-ascii 1)))
  (if (>= index (len items))
    none
    (as-max-len? (concat (concat (unwrap-panic (slice? items u0 index)) item)
                         (unwrap-panic (slice? items (+ index u1) (len items))))
                 N)))

(define-private (replace-at? (items (string-utf8 N)) (index uint) (item (string-utf8 1)))
  (if (>= index (len items))
    none
    (as-max-len? (concat (concat (unwrap-panic (slice? items u0 index)) item)
                         (unwrap-panic (slice? items (+ index u1) (len items))))
                 N)))

;; ---------------------------------------------------------------------------
;; Conversions
;; ---------------------------------------------------------------------------

;; The core has one conversion per pair of representations (`to-int`,
;; `to-uint`, `buff-to-uint-be`, `uint-to-buff-be`). Everything here is
;; derived from those.

(define-private (std-prepend-byte (acc (buff 16)) (byte (buff 1)))
  (unwrap-panic (as-max-len? (concat byte acc) u16)))

(define-private (std-reverse-16 (bytes (buff 16)))
  (fold-left std-prepend-byte 0x bytes))

(define-private (buff-to-uint-le (bytes (buff 16)))
  (buff-to-uint-be (std-reverse-16 bytes)))

;; Two's complement, sign-extended from the buffer's actual length, so 0xff
;; is -1 (legacy zero-padded a short buffer and gave 255; chapter 3.4). The
;; negative branch adds -2^(8L-1), computed without overflowing when L is 16.
(define-private (buff-to-int-be (bytes (buff 16)))
  (let ((magnitude (buff-to-uint-be bytes))
        (width (len bytes)))
    (if (is-eq width u0)
      0
      (let ((half (bit-shift-left u1 (- (* width u8) u1))))
        (if (< magnitude half)
          (to-int magnitude)
          (+ (to-int (- magnitude half))
             (- (- 0 (to-int (- half u1))) 1)))))))

(define-private (buff-to-int-le (bytes (buff 16)))
  (buff-to-int-be (std-reverse-16 bytes)))

;; |value| as a uint, including for the most negative int.
(define-private (std-magnitude (value int))
  (if (< value 0)
    (+ (to-uint (- -1 value)) u1)
    (to-uint value)))

;; Per-kind digit tables, selected by the text type.
```

```
(define-private (std-prepend-digit (digit uint) (text (string-ascii 40)))
  (unwrap-panic (as-max-len? (concat (unwrap-panic (element-at? "0123456789" digit)) text) u40)))

(define-private (std-prepend-digit (digit uint) (text (string-utf8 40)))
  (unwrap-panic (as-max-len? (concat (unwrap-panic (element-at? u"0123456789" digit)) text) u40)))

;; u340282366920938463463374607431768211455 has 39 digits.
(define-constant std-decimal-iterations
  (list true true true true true true true true true true true true true true
        true true true true true true true true true true true true true true
        true true true true true true true true true true true true true true true))

;; Emits one digit per step until the rest is zero; the first step always
;; emits, so zero renders as "0".
(define-private (std-decimal-step (acc { rest: uint, text: X }) (ignored bool))
  (if (and (is-eq (get rest acc) u0) (> (len (get text acc)) u0))
    acc
    { rest: (/ (get rest acc) u10),
      text: (std-prepend-digit (mod (get rest acc) u10) (get text acc)) }))

;; `X` is instantiated at the empty string of the result kind.
(define-private (std-render-decimal (value uint) (empty X))
  (get text (fold-left std-decimal-step { rest: value, text: empty }
              std-decimal-iterations)))

(define-private (std-render-signed (value int) (empty X) (minus Y))
  (if (< value 0)
    (unwrap-panic (as-max-len? (concat minus (std-render-decimal (std-magnitude value) empty))
u40))
    (std-render-decimal (to-uint value) empty)))

(define-private (int-to-ascii (value uint))
  (std-render-decimal value ""))

(define-private (int-to-ascii (value int))
  (std-render-signed value "" "-"))

(define-private (int-to-utf8 (value uint))
  (std-render-decimal value u""))

(define-private (int-to-utf8 (value int))
  (std-render-signed value u"" u"-"))

(define-private (std-digit-value (char (string-ascii 1)))
  (index-of? "0123456789" char))

(define-private (std-digit-value (char (string-utf8 1)))
  (index-of? u"0123456789" char))

(define-private (std-is-minus (char (string-ascii 1)))
  (is-eq char "-"))

(define-private (std-is-minus (char (string-utf8 1)))
  (is-eq char u"-"))

;; total * 10 + digit, or none if that exceeds the largest uint.
(define-private (std-push-digit (total uint) (digit uint))
  (if (or (> total u34028236692093846346337460743176821145)
          (and (is-eq total u34028236692093846346337460743176821145) (> digit u5)))
    none
    (some (+ (* total u10) digit))))

(define-private (std-parse-digit (acc (optional uint)) (char C))
  (match acc
    ((some total)
      (match (std-digit-value char)
        ((some digit) (std-push-digit total digit))
```

```
        (none none)))
    (none none)))

;; `S` is instantiated at string-ascii or string-utf8.
(define-private (string-to-uint? (text S))
  (if (is-eq (len text) u0)
    none
    (fold-left std-parse-digit (some u0) text)))

;; -magnitude, or none below the most negative int.
(define-private (std-negate-magnitude (magnitude uint))
  (if (is-eq magnitude u0)
    (some 0)
    (if (> magnitude u170141183460469231731687303715884105728)
      none
      (some (- (- 0 (to-int (- magnitude u1))) 1)))))

;; `S` is instantiated at string-ascii or string-utf8.
(define-private (string-to-int? (text S))
  (match (element-at? text u0)
    ((some first)
      (if (std-is-minus first)
        (match (string-to-uint? (unwrap-panic (slice? text u1 (len text))))
          ((some magnitude) (std-negate-magnitude magnitude))
          (none none))
        (match (string-to-uint? text)
          ((some magnitude)
            (if (> magnitude u170141183460469231731687303715884105727)
              none
              (some (to-int magnitude))))
          (none none))))
    (none none)))

;; `to-ascii?` renders the six kinds legacy accepts, with legacy's text and
;; result bounds, except that a principal's bound is 82, not 170, because
;; contract names are at most 40 characters. Only the string-utf8 overload
;; can fail. Principals go through the `principal-to-ascii` primitive,
;; because c32check is a chain encoding (chapter 3.5).

(define-private (to-ascii? (value int))
  (ok (int-to-ascii value)))

;; "u" and at most 39 digits.
(define-private (to-ascii? (value uint))
  (ok (unwrap-panic (as-max-len? (concat "u" (int-to-ascii value)) u40))))

(define-private (to-ascii? (value bool))
  (ok (if value "true" "false")))

(define-private (to-ascii? (value principal))
  (ok (principal-to-ascii value)))

;; Two lowercase hex digits.
(define-private (std-hex-byte (byte (buff 1)))
  (let ((number (buff-to-uint-be byte)))
    (concat (unwrap-panic (element-at? "0123456789abcdef" (/ number u16)))
            (unwrap-panic (element-at? "0123456789abcdef" (mod number u16))))))

;; "0x" and two digits per byte: the spread gives the bound 2 + N * 2.
(define-private (to-ascii? (value (buff N)))
  (ok (concat "0x" (map std-hex-byte value)...)))

;; The string-ascii character set (chapter 2.2), in the same order in both
;; kinds: 0x20 through 0x7E, then tab, line feed, carriage return.
(define-constant std-ascii-chars
  " !\"#$%&'()*+,-./0123456789:;<=>?
@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~\t\n\r")
```

```
(define-constant std-ascii-chars-utf8
  u" !\"#$%&'()*+,-./0123456789:;<=>?
@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~\t\n\r")

(define-private (std-char-to-ascii (char (string-utf8 1)))
  (match (index-of? std-ascii-chars-utf8 char)
    ((some index) (element-at? std-ascii-chars index))
    (none none)))

(define-private (std-char-or-empty (char (optional (string-ascii 1))))
  (default-to "" char))

;; (err u1) if any character has no string-ascii form. The spread gives the
;; bound N * 1.
(define-private (to-ascii? (value (string-utf8 N)))
  (let ((chars (map std-char-to-ascii value)))
    (if (is-eq (len (filter is-some chars)) (len chars))
      (ok (concat "" (map std-char-or-empty chars)...))
      (err u1))))

;; ---------------------------------------------------------------------------
;; Principals
;; ---------------------------------------------------------------------------

;; True if the principal's version byte is for the current network, whether it
;; is a standard or a contract principal. This is what legacy `is-standard`
;; tested, under a name that said otherwise.
(define-private (is-current-network (who principal))
  (is-ok (principal-destruct? who)))

;; True if the principal is a standard principal, not a contract principal,
;; on any network.
(define-private (is-standard (who principal))
  (match (principal-destruct? who)
    ((ok parts) (is-none (get name parts)))
    ((err parts) (is-none (get name parts)))))

;; Version bytes for single-signature standard principals.
(define-constant mainnet-p2pkh-version 0x16)
(define-constant testnet-p2pkh-version 0x1a)

;; Derive the standard principal for a compressed public key on the current
;; network. Returns (err u1) if the key is not 33 bytes.
(define-private (principal-of? (public-key (buff 33)))
  (if (not (is-eq (len public-key) u33))
    (err u1)
    (match (principal-construct?
             (if (get is-mainnet (get-current-context))
               mainnet-p2pkh-version
               testnet-p2pkh-version)
             (hash160 public-key)
             none)
      ((ok who) (ok who))
      ((err ignored) (err u1)))))

;; ---------------------------------------------------------------------------
;; Cryptography
;; ---------------------------------------------------------------------------

(define-private (std-double-sha256 (bytes (buff 64)))
  (sha256 (sha256 bytes)))

;; Depth of a Bitcoin Merkle tree over `count` leaves (count >= 1).
(define-private (std-merkle-depth (count uint))
  (if (<= count u1)
    u0
    (+ (log2 (- count u1)) u1)))
```

```
;; One level up the tree. `width` is the number of nodes in the current row.
;; A left node that is last in its row is paired with itself, so its sibling
;; must equal it.
(define-private (std-merkle-step (acc { hash: (buff 32), index: uint, valid: bool, width: uint })
                                 (sibling (buff 32)))
  (let ((current (get hash acc))
        (position (get index acc))
        (right-child (is-eq (mod position u2) u1)))
    { hash: (if right-child
              (std-double-sha256 (concat sibling current))
              (std-double-sha256 (concat current sibling))),
      index: (/ position u2),
      valid: (and (get valid acc)
                  (or right-child
                      (not (is-eq (+ position u1) (get width acc)))
                      (is-eq sibling current))),
      width: (+ (/ (get width acc) u2) (mod (get width acc) u2)) }))

;; Bitcoin-style inclusion proof; see chapter 4.2 for the full contract. `false` for
;; an index not below the count, a path length other than the tree depth, a
;; failed duplication check, or a root mismatch.
(define-private (verify-merkle-proof (leaf (buff 32)) (root (buff 32))
                                     (index uint) (count uint)
                                     (proof (list 24 (buff 32))))
  (if (or (>= index count) (not (is-eq (len proof) (std-merkle-depth count))))
    false
    (let ((final (fold-left std-merkle-step
                  { hash: leaf, index: index, valid: true, width: count }
                  proof)))
      (and (get valid final) (is-eq (get hash final) root)))))

;; ---------------------------------------------------------------------------
;; Storage
;; ---------------------------------------------------------------------------

;; Insert only if absent. Returns the existing value, and writes nothing, if
;; the key already has one; `none` if the value was inserted. Private, so
;; `key` and `value` are both evaluated before the call.
(define-private (map-insert? (target (map K V)) (key K) (value V))
  (match (map-get? target key)
    ((some existing) (some existing))
    (none (begin (map-set target key value) none))))

;; ---------------------------------------------------------------------------
;; Assets
;; ---------------------------------------------------------------------------

(define-private (stx-get-balance (owner principal))
  (get unlocked (stx-account owner)))
```